



FIRAT ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ
Yazılım Mühendisliği Bölümü

YMH219 – NESNE TABANLI PROGRAMLAMA
Dersi Proje Uygulaması ve Dokümantasyonu

Project Tracker - Proje Yönetim Sistemi

Geliştiren
240540000 Bilal ABİÇ

Proje Yürütücüleri
Dr. Öğr. Üyesi V. Cem BAYDOĞAN
Arş. Gör. Hüseyin Alperen DAĞDÖGEN

OCAK – 2026

ÖNSÖZ VE TEŞEKKÜR

Hayatım boyunca ve bu çalışma süresince desteklerini esirgemeyen ailem ve arkadaşlarıma teşekkürü bir borç bilirim. Bu projeyi gerçekleştirme aşamasında yararlandığım her kaynağı kaynaklar kısmında bildirdiğimi taahhüt ederim.

Bilal ABİÇ

İÇİNDEKİLER

1. GİRİŞ	7
1.1. Projenin Tanıtılması	7
1.2. Projenin Amacı	7
1.3. Projenin Kapsamı	7
1.3.1. Projede Yapılanlar:.....	7
1.3.2. Geliştirilmesi Planlanan Özellikler:	7
1.4. Kullanılacak Teknolojiler	8
1.4.1. Backend Teknolojileri	8
1.4.2. Frontend Teknolojileri.....	8
1.4.3. Entegrasyonlar	8
2. ÇÖZÜMLEME.....	8
2.1. Mevcut Projelerin Eksiklikleri ve Bizim Farkımız.....	8
2.1.1. Mevcut Araçların Eksiklikleri	8
2.1.2. Project Tracker'ın Farkları.....	9
2.2. Arayüz Gerekliliği	9
2.2.1. Neden Windows Forms?	9
2.2.2. Arayüz Tasarım Prensipleri.....	9
2.3. Sistemin Kullanıcıları	9
2.3.1. Kullanıcı Kayıt Akışı	10
2.4. İşlevsel İhtiyaçlar (Olmazsa Olmazlar)	10
2.4.1. Kullanıcı Yönetimi	10
2.4.2. Proje Yönetimi	10
2.4.3. Görev Yönetimi.....	10
2.4.4. Takım Yönetimi	10
2.4.5. Raporlama	10
2.5. İşlevsel Olmayan İhtiyaçlar	10
3. TASARIM.....	11
3.1. Sistem Tasarımı - Proje Mimarisi.....	11
3.1.1. Katmanlı Mimari (5 Katman).....	11
3.1.2. Neden Bu Mimariyi Seçtim?	12
3.1.3. Katmanlı Mimari Avantajları	12
3.1.4. Design Patterns Kullanımı	12
3.2. Veri Tasarımı - Tablo İlişki Sistemi.....	12
3.2.1. Veritabanı Mimarisi ve Yaklaşımı	12

3.2.2.	Tablo Listesi (18 Tablo).....	12
3.2.3.	Modül 1: Kimlik ve Organizasyon (Identity & Core).....	13
3.2.4.	Modül 2: Proje Yönetimi (Project Management).....	14
3.2.5.	Modül 3: Git Entegrasyonu (Code & Sync).....	15
3.2.6.	Modüller Arası Entegrasyon Tablosu	15
3.2.7.	Tasarım Kararları (Neyi Neden Yaptım?).....	16
3.2.8.	Enum Tanımları.....	16
3.3.	Süreç Modeli - Hangisi ve Neden Seçtim?.....	16
3.3.1.	Seçilen Model: Artırmalı Geliştirme (Incremental Development)	17
3.3.2.	Artırım Planı.....	17
3.3.3.	GANTT İş Akış Diyagramı.....	18
3.4.	UML Diyagramları	19
3.4.1.	Kimlik ve Organizasyon Katmanı (Identity & Core Layer)	19
3.4.2.	Proje Yönetim Katmanı (Project Management Layer)	19
3.4.3.	Git Entegrasyon Katmanı (Git Integration Layer)	20
3.4.4.	Genel Sistem Mimarisi ve Modüller Arası İlişkiler (Tam Görünüm).....	21
3.5.	Use Case Diyagramı	22
3.5.1.	Use Case Diyagramı (Kullanım Senaryoları).....	22
3.5.2.	Modül Bazlı Detaylı Senaryolar.....	23
3.5.3.	Use Case Senaryo Listesi	24
3.6.	Arayüz Tasarımı - Modüllerin ve Formların Tanıtımı	24
3.6.1.	Giriş Modülü	25
3.6.2.	Dashboard Modülü.....	26
3.6.3.	Proje Modülü.....	28
3.6.4.	Görev Modülü	30
3.6.5.	Takım Modülü.....	32
3.6.6.	GitHub Modülü	36
3.6.7.	Raporlama Modülü.....	38
3.6.8.	Ayarlar Modülü	40
3.6.9.	Hata Yönetimi	41
4.	KODLAMA	42
4.1.	Programlama Dili - Neden C#?	42
4.1.1.	C# Seçim Nedenleri	42
4.1.2.	C# 12.0 Özellikleri Kullanımı.....	42
4.2.	Modüller	42

4.2.1.	Core Modülü (ProjectTracker.Core)	42
4.2.2.	Data Modülü (ProjectTracker.Data).....	43
4.2.3.	Business Modülü (ProjectTracker.Business)	44
4.2.4.	UI Modülü (ProjectTracker.UI)	45
4.2.5.	API Modülü (ProjectTracker.API).....	45
4.3.	Kod Stilleri	46
4.3.1.	Naming Conventions.....	46
4.3.2.	Kod Organizasyonu.....	46
4.3.3.	SOLID Prensipleri Uygulaması	47
4.4.	Program Karmaşıklığı.....	47
4.4.1.	İşlev Nokta Analizi (Function Point Analysis)	47
4.4.2.	Teknik Karmaşıklık Faktörü (TKF)	47
4.4.3.	Ayarlanmış İşlev Noktası	48
4.4.4.	Kod Satır Sayısı Tahmini	48
4.4.5.	Karmaşıklık Değerlendirmesi	48
4.4.6.	Karmaşıklığı Artıran Faktörler.....	48
4.4.7.	Analiz Sonuçlarının Değerlendirilmesi(Google Gemini AI)	48
4.5.	Akıllı Algoritmalar	48
4.5.1.	Risk Skoru Hesaplama Algoritması	48
4.5.2.	Commit-Task Eşleştirme Algoritması.....	49
5.	DOĞRULAMA VE GEÇERLEME.....	49
5.1.	Arayüz Çalışıyor mu? (Fonksiyonel Testler)	49
5.1.1.	Giriş Modülü Testleri	49
5.1.2.	Dashboard Modülü Testleri.....	50
5.1.3.	Proje Modülü Testleri.....	50
5.1.4.	Görev Modülü Testleri	50
5.1.5.	Takım Modülü Testleri.....	50
5.1.6.	GitHub Modülü Testleri	50
5.1.7.	Raporlama Modülü Testleri.....	50
5.2.	Unit Test Sonuçları.....	51
5.2.1.	Test Özeti	51
5.2.2.	Servis Bazlı Test Dağılımı	51
5.2.3.	Test Teknolojileri	51
5.2.4.	Örnek Test Kodu	51
5.3.	Kod Kalitesi - SonarQube Analizi.....	52

5.3.1.	SonarQube Analiz Metrikleri	52
5.3.2.	Kod Kalitesi ve Güvenlik Özellikleri	53
6.	SONUÇ	53
6.1.	Projenin Genel Değerlendirmesi	53
6.1.1.	Proje Başarıları ve Güçlü Yönler	53
6.1.2.	Kısıtlar ve Eksiklikler.....	53
6.2.	Projenin Bireysel Katkıları	54
6.2.1.	Kazanılan Teknik Beceriler.....	54
6.2.2.	Soft Skills (Kişisel Gelişim).....	54
6.3.	Proje Metrikleri Özeti	54
6.4.	Gelecek Planları ve Yol Haritası	54
7.	KAYNAKLAR.....	55
7.1.	Kitaplar	55
7.2.	Online Dokümantasyon	55
7.3.	Proje Bağlantıları ve Teknik Diyagramlar.....	55
7.4.	Eğitim Videoları ve Görsel Kaynaklar	55
7.5.	Kullanılan NuGet Paketleri	56

1. GİRİŞ

1.1. Projenin Tanıtılması

Project Tracker, yazılım geliştirme ekiplerinin günlük işlerini kolaylaştırmak için tasarladığım kapsamlı bir proje yönetim sistemi. Aslında bu projeyi geliştirirken kendi ihtiyaçlarımdan yola çıktım - bir projede çalışırken görevlerin takibi, ekip üyeleriyle koordinasyon ve ilerlemenin raporlanması her zaman zorlayıcı olabiliyor.

Bu sistem sayesinde:

- Projelerinizi tek bir yerden yönetebilirsiniz
- Görevleri Kanban board üzerinde sürükle-bırak ile organize edebilirsiniz
- Takım arkadaşlarınızı e-posta ile davet edebilirsiniz
- GitHub repository'nizi bağlayarak commit'lerinizi takip edebilirsiniz
- Detaylı raporlar alarak ilerlemenizi görebilirsiniz

Projenin en ilginç özelliklerinden biri, GitHub'dan çekilen commit mesajlarını analiz ederek hangi görevle ilişkili olduğunu otomatik olarak eşleştiren akıllı algoritma. Ayrıca proje risk skorunu hesaplayan bir algoritma da mevcut.

1.2. Projenin Amacı

Bu projeyi geliştirirken birkaç temel amacım vardı:

Eğitim Açısından:

- Nesne tabanlı programlama prensiplerini (SOLID) gerçek bir projede uygulamak
- Katmanlı mimari tasarımını öğrenmek ve deneyimlemek
- Design pattern'leri (Repository, Unit of Work, Dependency Injection) kullanmak

Pratik Açıdan:

- Yazılım ekiplerinin gerçek ihtiyaçlarına cevap veren bir ürün ortaya koymak
- GitHub API, SMTP gibi harici servislerle entegrasyon deneyimi kazanmak
- Entity Framework Core ile veritabanı yönetimini öğrenmek

1.3. Projenin Kapsamı

1.3.1. Projede Yapılanlar:

- ✓ Kullanıcı giriş/kayıt sistemi (4 farklı rol: Admin, ProjectManager, Developer, Pending)
- ✓ Proje oluşturma, düzenleme, silme ve takip
- ✓ Görev yönetimi (hem liste hem Kanban görünümü)
- ✓ Takım oluşturma ve üye yönetimi
- ✓ E-posta ile davet gönderme (Gmail SMTP)
- ✓ Web üzerinden davet kabul etme (GitHub Pages'da barındırılan site)
- ✓ GitHub repository bağlama ve commit senkronizasyonu
- ✓ Detaylı raporlama ve PDF/Excel export
- ✓ Risk hesaplama ve audit log sistemi
- ✓ 177 adet unit test ile kod kalitesi güvencesi

1.3.2. Geliştirilmesi Planlanan Özellikler:

- Mobil uygulama
- Real-time bildirimler (SignalR)
- Dosya yükleme özelliği

- Çoklu dil desteği

1.4. Kullanılacak Teknolojiler

1.4.1. Backend Teknolojileri

Teknoloji	Versiyon	Ne İçin Kullandım
.NET	8.0 LTS	Ana framework - Microsoft'un en güncel ve stabil sürümü
C#	12.0	Programlama dili - modern syntax özellikleri
Entity Framework Core	8.0	Veritabanı işlemleri için ORM
SQL Server	2022	Ana veritabanı
ASP.NET Core	8.0	Web API için
AutoMapper	12.0	Entity-DTO dönüşümleri
FluentValidation	11.0	Kullanıcı girişi doğrulama
BCrypt.Net	4.0	Şifreleri güvenli şekilde hashlemek için

1.4.2. Frontend Teknolojileri

Teknoloji	Versiyon	Ne İçin Kullandım
Windows Form	.NET 8	Masaüstü Uygulama Arayüzü
DevExpress WinForms	25.1.7	Profesyonel görünümlü UI kontrolleri
HTML5/CSS3/JavaScript	-	Web davet sayfası

1.4.3. Entegrasyonlar

Servis	Kullanım Amacı
GitHub REST API (Octokit)	Commit ve repository bilgilerini çekmek
Gmail SMTP	E-posta bildirimleri göndermek
Plesk Remote API	Web üzerinden davet kabul sistemi

2. ÇÖZÜMLEME

2.1. Mevcut Projelerin Eksiklikleri ve Bizim Farkımız

Piyasadaki mevcut proje yönetim araçlarını incelediğimde bazı eksiklikler dikkatimi çekti:

2.1.1. Mevcut Araçların Eksiklikleri

Araç	Eksiklik
Jira	Çok karmaşık arayüz, öğrenme eğrisi yüksek, pahalı lisans
Trello	Sadece Kanban, raporlama yok, GitHub entegrasyonu sınırlı

Asana	Masaüstü uygulaması yok, offline çalışmıyor
Monday.com	Pahalı, gereksiz özellik kalabalığı
GitHub Projects	Sadece GitHub'a bağımlı, bağımsız proje yönetimi yok

2.1.2. Project Tracker'ın Farkları

Özellik	Bizim Çözümümüz
Basit Arayüz	DevExpress ile modern ama sade tasarım
Hem Liste Hem Kanban	Kullanıcı tercihine göre görünüm değiştirme
GitHub Entegrasyonu	Commit'leri otomatik çekme ve görevlerle eşleştirme
Akıllı Algoritmalar	Risk hesaplama, commit-task eşleştirme
Offline Çalışma	Masaüstü uygulaması, yerel veritabanı
Ücretsiz	Açık kaynak, lisans ücreti yok
Açık Kaynak Kodlu Yapı	GitHub üzerinden topluluk katılımına açık, şeffaf ve geliştirilebilir mimari

2.2. Arayüz Gerekliliği

2.2.1. Neden Windows Forms?

- Masaüstü Deneyimi: Kullanıcılar tarayıcı açmadan doğrudan uygulamayı çalıştırabilir
- Performans: Web uygulamalarına göre daha hızlı yanıt süresi
- Offline Çalışma: İnternet bağlantısı olmadan da çalışabilme
- DevExpress Kontrolleri: Profesyonel görünümlü, zengin özellikli UI bileşenleri
- Ders Gereksinimleri: Nesne Tabanlı Programlama dersi Windows Forms odaklı

2.2.2. Arayüz Tasarım Prensipleri

Prensip	Uygulama
Tutarlılık	Tüm ekranlarda aynı renk paleti ve stil
Basitlik	Gereksiz karmaşıklıktan kaçınma
Geri Bildirim	Her işlem sonrası kullanıcıya bilgi verme
Erişilebilirlik	Yüksek kontrast, okunabilir fontlar

2.3. Sistemin Kullanıcıları

Sistemi tasarlarken farklı kullanıcı tiplerinin farklı ihtiyaçları olacağını düşündüm. Bu yüzden 4 farklı rol tanımladım:

Rol	Açıklama	Neler Yapabilir?
Admin	Sistem yöneticisi	Her şeyi yapabilir - kullanıcı onaylama, rol değiştirme, tüm verilere erişim
ProjectManager	Proje yöneticisi	Proje ve takım oluşturma, görev atama, raporları görüntüleme
Developer	Geliştirici	Kendine atanan görevleri görme ve güncelleme, yorum ekleme
Pending	Onay bekleyen	Sadece bekleme ekranını görür, admin onayı bekler

2.3.1. Kullanıcı Kayıt Akışı

İki farklı şekilde sisteme kayıt olunabilir:

- Direkt Kayıt: Kullanıcı kayıt formunu doldurur → Pending rolüyle kaydedilir → Admin onaylar → Developer veya ProjectManager rolü atanır
- Davetli Kayıt: Takım sahibi e-posta ile davet gönderir → Kullanıcı web'den daveti kabul eder → Kayıt formunu doldurur → Davetteki rol otomatik atanır

2.4. İşlevsel İhtiyaçlar (Olmazsa Olmazlar)

2.4.1. Kullanıcı Yönetimi

- ✓ Kullanıcılar sisteme kayıt olabilmeli
- ✓ Kullanıcılar giriş yapabilmeli
- ✓ Admin, bekleyen kullanıcıları onaylayabilmeli
- ✓ Şifreler güvenli şekilde (BCrypt) saklanmalı

2.4.2. Proje Yönetimi

- ✓ Yeni proje oluşturulabilmeli
- ✓ Proje bilgileri güncellenebilmeli
- ✓ Proje silinebilmeli
- ✓ Proje bir takıma atanabilmeli
- ✓ Projeye GitHub repository bağlanabilmeli
- ✓ Proje risk skoru otomatik hesaplanmalı

2.4.3. Görev Yönetimi

- ✓ Görev oluşturulabilmeli
- ✓ Görev bir kullanıcıya atanabilmeli
- ✓ Görev durumu değiştirilebilmeli (Pending → InProgress → Completed)
- ✓ Kanban board görünümü olmalı
- ✓ Görev atandığında e-posta bildirimi gitmeli

2.4.4. Takım Yönetimi

- ✓ Takım oluşturulabilmeli
- ✓ Takıma üye eklenebilmeli
- ✓ E-posta ile davet gönderilebilmeli
- ✓ Davet web üzerinden kabul edilebilmeli

2.4.5. Raporlama

- ✓ Proje bazlı rapor alınabilmeli
- ✓ PDF ve Excel formatında export yapılabilmesi
- ✓ Tüm işlemler audit log'a kaydedilmeli

2.5. İşlevsel Olmayan İhtiyaçlar

Kategori	Gereksinim	Nasıl Sağladım?
Performans	Sayfalar 2 saniyeden kısa sürede yüklenmeli	Async/await kullanımı, lazy loading
Güvenlik	Şifreler güvenli saklanmalı	BCrypt ile hashleme
Güvenlik	GitHub token'ları şifreli tutulmalı	AES şifreleme

Kullanılabilirlik	Modern ve kullanıcı dostu arayüz	DevExpress kontrolleri
Bakım	Kod kolay bakım yapılabilir olmalı	Katmanlı mimari, SOLID prensipleri
Ölçeklenebilirlik	Yeni özellikler kolayca eklenebilmeli	Generic repository, DI
Güvenilirlik	Tüm işlemler kayıt altına alınmalı	Audit log sistemi

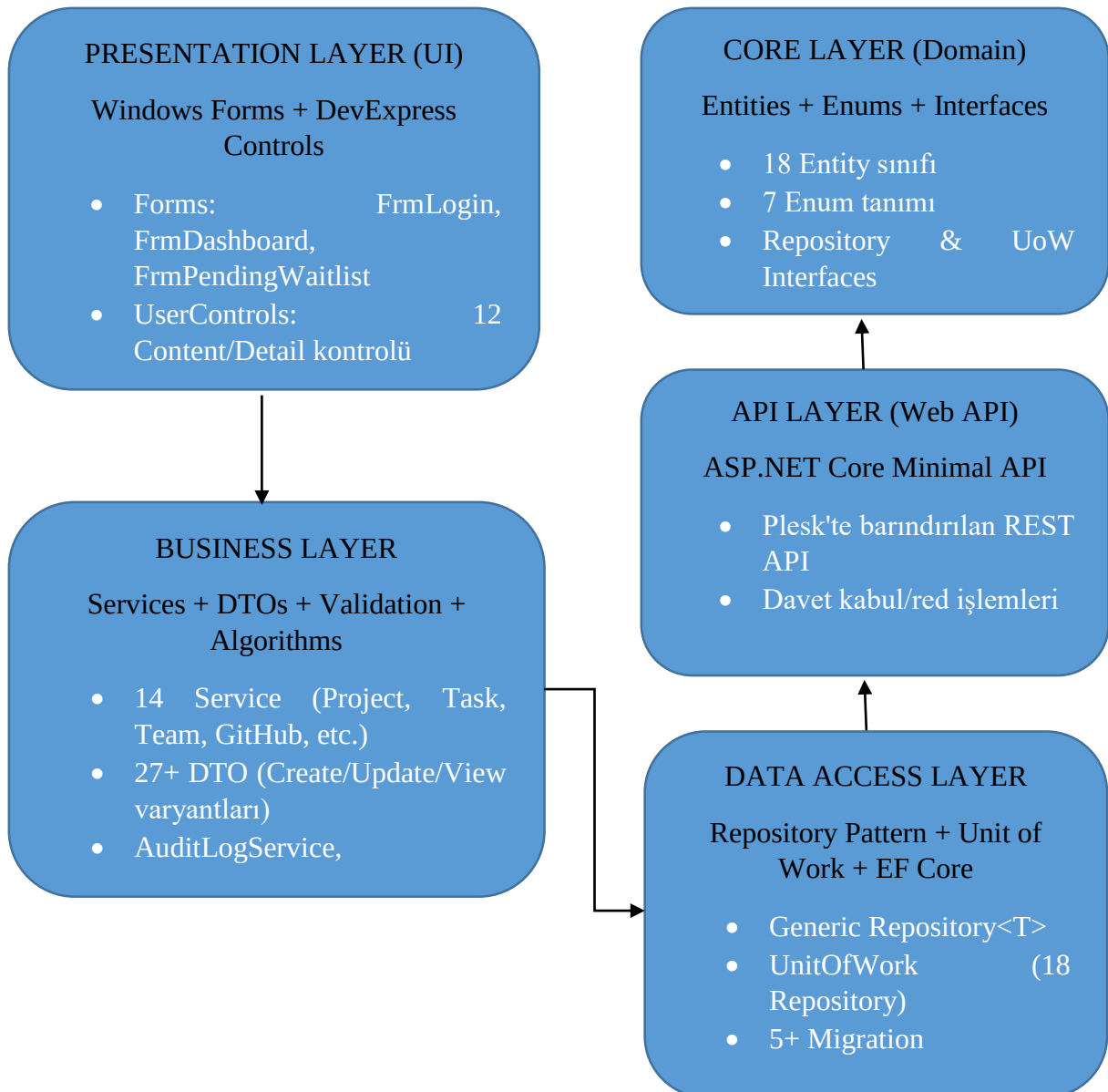
3. TASARIM

3.1. Sistem Tasarımı - Proje Mimarisi

3.1.1. Katmanlı Mimari (5 Katman)

Projeyi 5 katmanlı enterprise mimari ile tasarladım. Bu yaklaşımın avantajları:

- Her katmanın tek bir sorumluluğu var (Single Responsibility)
- Katmanlar birbirinden bağımsız test edilebilir
- Değişiklikler sadece ilgili katmanı etkiler
- Kod tekrarı minimize edilir



3.1.2. Neden Bu Mimariyi Seçtim?

Alternatif	Neden Seçmedim
Monolitik	Bakımı zor, test edilemez, ölçeklenemez
Microservices	Proje boyutu için fazla karmaşık
MVC	Windows Forms için uygun değil
MVVM	WPF için daha uygun, WinForms'ta karmaşık

3.1.3. Katmanlı Mimari Avantajları

- Separation of Concerns (Kaygıların Ayrılması)
- Dependency Injection ile loose coupling
- Unit test yazılabilirlik
- Kod tekrarının önlenmesi
- Bakım kolaylığı

3.1.4. Design Patterns Kullanımı

Pattern	Kullanım Yeri	Açıklama
Repository Pattern	Data Layer	Veri erişim soyutlaması
Unit of Work	Data Layer	Transaction yönetimi
Dependency Injection	Tüm katmanlar	Loose coupling
DTO Pattern	Business Layer	Katmanlar arası veri transferi
Service Pattern	Business Layer	İş mantığı kapsülleme
Singleton	UI Layer	SessionManager
Factory	Data Layer	DbContext oluşturma

3.2. Veri Tasarımı - Tablo İlişki Sistemi

3.2.1. Veritabanı Mimarisi ve Yaklaşımı

Projede Dual-Database (Çift Veritabanı) mimarisi tercih edilmiştir. Bu hibrit yaklaşım, masaüstü uygulaması (Local) ile web tabanlı davet sisteminin (Remote) haberleşme ve güvenlik kısıtlarını aşmak için geliştirilmiştir:

1. **Yerel SQL Server (Ana Veritabanı):** Uygulamanın temel verilerini barındırır. Kullanıcılar, projeler, görevler, GitHub verileri ve sistem logları burada tutulur (17 Tablo).
2. **Plesk Remote Database (Davet Veritabanı):** Sadece dış dünyaya açık olması gereken davet işlemleri için kullanılır. Web API güvenli bir şekilde buraya erişir (1 Tablo).

Toplam 18 tablodan oluşan kapsamlı veritabanı şeması, yönetim kolaylığı ve anlaşılabilirlik açısından mantıksal olarak 3 Ana Modüle ayrılmıştır. Aşağıdaki bölümlerde bu modüllerin varlık-ilişki (ER) diyagramları detaylandırılmıştır.

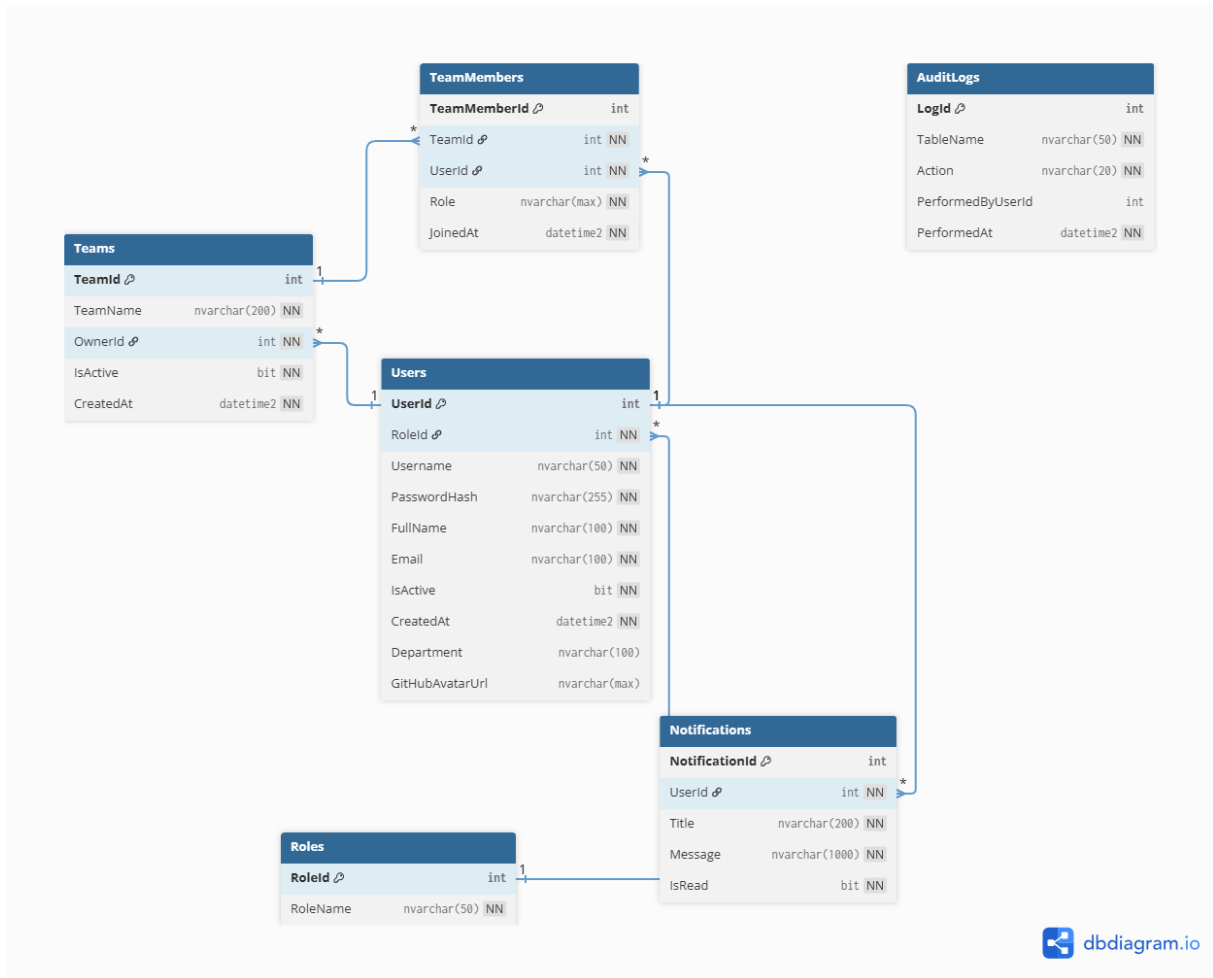
3.2.2. Tablo Listesi (18 Tablo)

Tablo	Açıklama	İlişkiler
Users	Kullanıcı bilgileri	Roles (N:1)
Roles	Sistem rolleri (4 adet)	Users (1:N)
Teams	Takım bilgileri	Users (N:1), Projects (1:N)
TeamMembers	Takım-Kullanıcı ilişkisi	Teams (N:1), Users (N:1)
TeamInvitations	Takım davetleri	Teams (N:1), Users (N:1)
Projects	Proje bilgileri	Teams (N:1), Tasks (1:N)
Tasks	Görev bilgileri	Projects (N:1), Users (N:1)

TaskComments	Görev yorumları	Tasks (N:1), Users (N:1)
ProjectTeamMembers	Proje-Kullanıcı ilişkisi	Projects (N:1), Users (N:1)
ProjectRisks	Risk kayıtları	Projects (N:1)
ProjectSnapshots	Günlük anlık görüntüler	Projects (N:1)
Notifications	Bildirimler	Users (N:1)
TimeEntries	Zaman kayıtları	Tasks (N:1), Users (N:1)
AuditLogs	Denetim kayıtları	-
GitHubTokens	GitHub token havuzu	Users (N:1)
GitRepositories	GitHub repo bağlantıları	Projects (1:1)
GitCommits	Commit cache	GitRepositories (N:1), Tasks (N:1)
GitFileChanges	Dosya değişiklikleri	GitCommits (N:1)

3.2.3. Modül 1: Kimlik ve Organizasyon (Identity & Core)

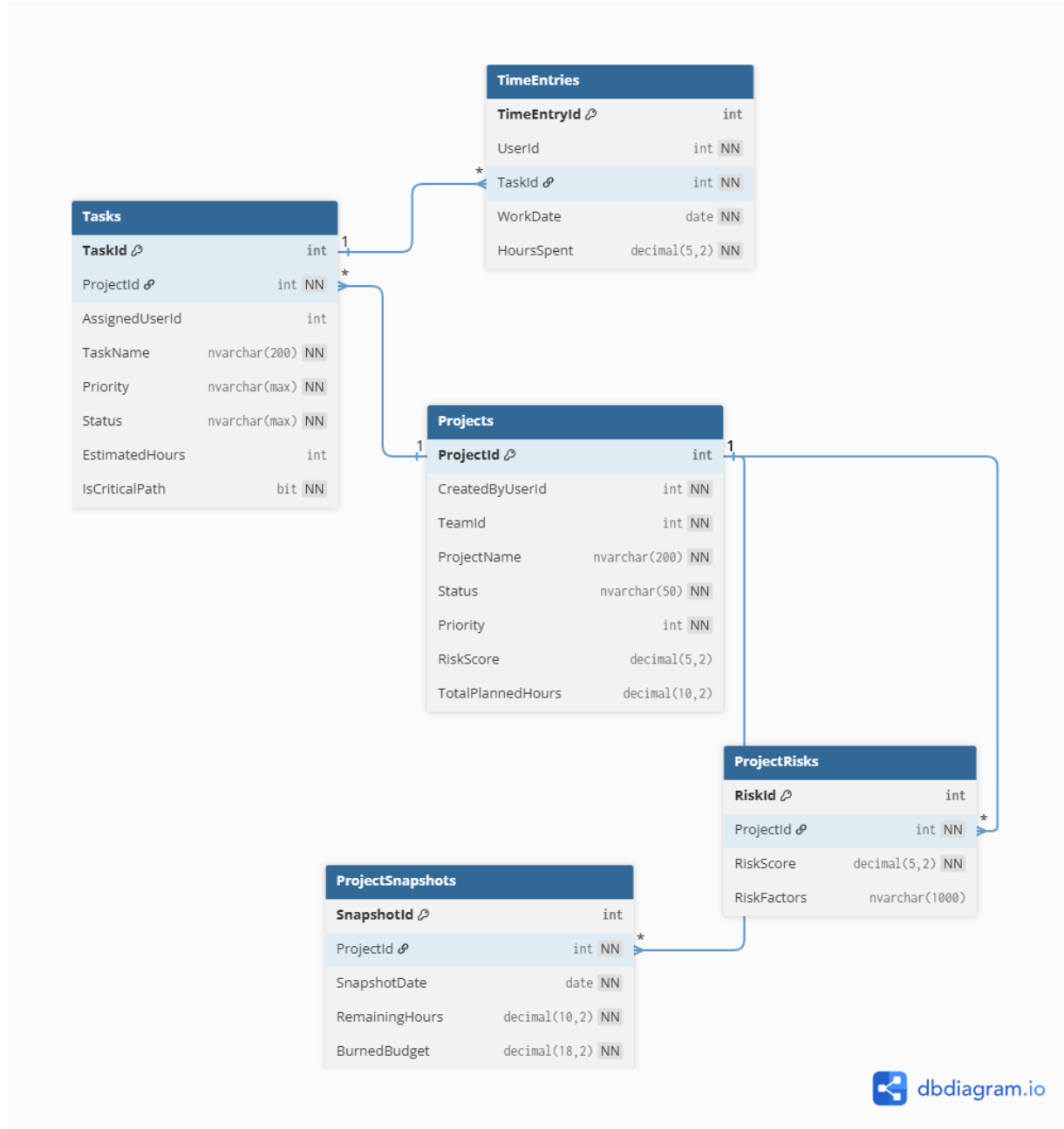
Sistemin "Kim?" sorusunu cevaplayan omurga kısmıdır. Kullanıcı yönetimi, yetkilendirme (RBAC), takım hiyerarşisi ve güvenlik logları bu modülde yer alır. Şekil 3.1'de bu modülün ilişki şeması görülmektedir.



Modül İlişkisi: Bu modül, sistemin merkezidir. Diğer tüm modüller, buradaki **Users** (Kullanıcılar) ve **Teams** (Takımlar) tablolarını referans alarak veri bütünlüğünü sağlar.

3.2.4. Modül 2: Proje Yönetimi (Project Management)

Uygulamanın ana iş mantığının (Business Logic) döndüğü katmandır. Proje planlama, görev takibi, risk analizi, bütçe yönetimi ve zaman takibi tablolarını içerir. Şekil 3.2'de proje yönetim akışını sağlayan tablolar gösterilmiştir.

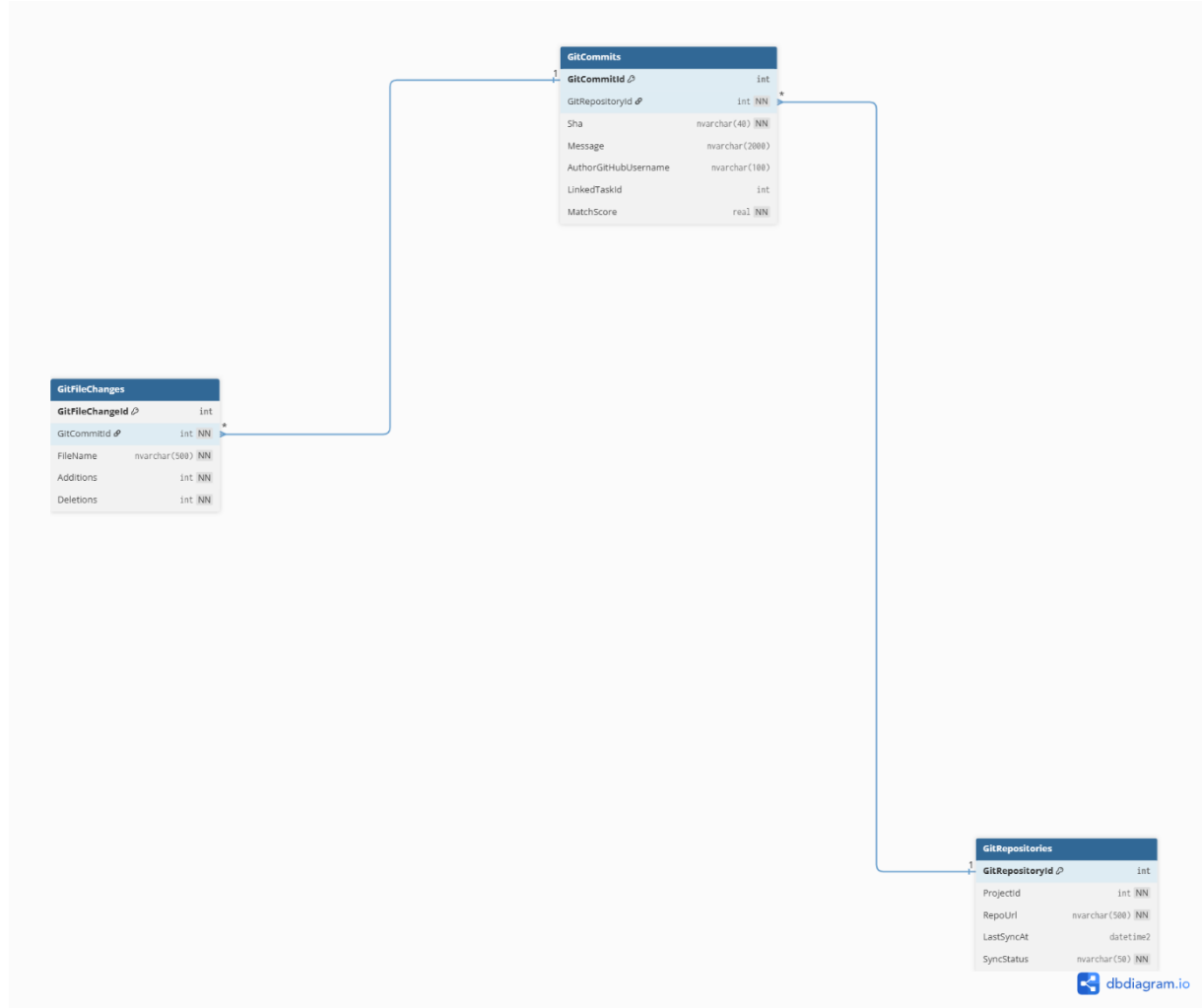


Dış Bağlantılar:

- **Projects** tablosu, **Modül 1**'deki **Teams** tablosuna bağlanarak projenin sahibini belirler.
- **Tasks** tablosu, **Modül 1**'deki **Users** tablosuna bağlanarak görevi yapan kişiyi belirler.

3.2.5. Modül 3: Git Entegrasyonu (Code & Sync)

Proje takibinin teknik tarafını ve kaynak kod yönetimini üstlenir. GitHub API'den çekilen repository, commit ve dosya değişiklik verileri burada önbelleğe (cache) alınır. Şekil 3.3'te Git entegrasyon şeması yer almaktadır.



Dış Bağlantılar:

- GitRepositories tablosu, **Modül 2**'deki Projects tablosuna bağlanır (Her projenin bir deposu vardır).
- GitCommits tablosu, **Modül 2**'deki Tasks tablosuna bağlanarak kodun hangi görev için yazıldığını takip eder (Traceability).

3.2.6. Modüller Arası Entegrasyon Tablosu

Bu üç parçalı yapının bir bütün olarak nasıl çalıştığını ve veri akışını sağlayan temel bağlantı noktaları aşağıdaki tabloda özetlenmiştir:

Bağlantı Tipi	Kaynak Modül	Hedef Modül	İlişki Açıklaması
Sahiplik	Modül 2 (Projects)	Modül 1 (Teams)	Her proje mutlaka bir takıma aittir. (Projects.TeamId)

İş Atama	Modül 2 (Tasks)	Modül 1 (Users)	Görevler sistemdeki kayıtlı kullanıcılara atanır. (Tasks.AssignedUserId)
Repo Bağlama	Modül 3 (Git)	Modül 2 (Projects)	Bir proje teknik olarak bir GitHub reposuna bağlanır. (GitRepositories.ProjectId)
İzlenebilirlik	Modül 3 (Git)	Modül 2 (Tasks)	Akıllı algoritma, commit mesajlarını analiz ederek ilgili Task ile eşleştirir. (GitCommits.LinkedTaskId)

3.2.7. Tasarım Kararları (Neyi Neden Yaptım?)

Veritabanı tasarımında performans ve ölçeklenebilirlik gözetilerek aşağıdaki kritik kararlar alınmıştır:

Tasarım Kararı	Neden?
Roles Tablosu	Sabit Enum yerine tablo yapısı kullanılarak, ileride sisteme yeni rollerin dinamik olarak eklenebilmesi sağlandı.
GitCommits Cache	GitHub API'nin saatlik istek sınırına (rate limit) takılmamak için commit verileri yerel veritabanına kaydedildi.
ProjectSnapshots	"Burndown Chart" raporunu anlık hesaplamak yerine, her gece projenin durumunu kaydeden bir özet tablosu oluşturuldu.
AuditLogs	Güvenlik ve hata takibi için kritik tablolardaki tüm INSERT/UPDATE/DELETE işlemleri log altına alındı.

3.2.8. Enum Tanımları

Veritabanında sayısal (int) olarak tutulan durum bilgilerinin kod tarafındaki karşılıkları aşağıdadır:

```
// Proje Durumları
public enum ProjectStatus { Planned = 1, Active = 2, OnHold = 3, Completed = 4, Cancelled = 5 }

// Görev Durumları
public enum TaskStatus { Pending = 1, InProgress = 2, Completed = 3, Blocked = 4 }

// Öncelik Seviyeleri
public enum Priority { Low = 1, Medium = 2, High = 3, Critical = 4 }

// Takım İçi Roller
public enum TeamRole { Owner = 1, Admin = 2, Developer = 3 }
```

Not: Veritabanı Entity Relationship (ER) diyagramının tamamına ve yüksek çözünürlüklü haline **kaynaklar kısmından ulaşabilirsiniz.**

3.3. Süreç Modeli - Hangisi ve Neden Seçtim?

3.3.1. Seçilen Model: Artırmalı Geliştirme (Incremental Development)

Bu proje için Artırmalı Geliştirme (Incremental Development) modelini seçtim. Bu modelde, proje küçük parçalara bölünür ve her artırımda (increment) üzerine koyarak çalışan bir ürün ortaya çıkarılır.

Neden Artırmalı Model?

Aşağıdaki tabloda, değerlendirilen diğer modeller ve neden seçilmedikleri özetlenmiştir:

Alternatif Model	Neden Seçmedim?
Şelale (Waterfall)	Geri dönüş zor, değişikliklere kapalı ve esnek değil.
Spiral	Risk analizi odaklı olduğu için küçük ölçekli projeler için fazla karmaşık ve maliyetli.
Agile/Scrum	Tek kişilik bir proje olduğu için günlük stand-up ve sprint planlaması gibi ritüeller gereksiz zaman kaybı yaratır.
RAD	Prototip odaklı olduğu için mimari yapı (Clean Architecture) zayıf kalabilir.

Artırmalı Modelin Avantajları:

- Her artırım sonunda çalışan somut bir ürün elde edilir.
- Erken aşamada geri bildirim alınabilir ve hatalar düzeltilebilir.
- Riskler projenin sonuna bırakılmadan erken tespit edilir.
- Değişen gereksinimlere ve değişikliklere açıktır.

3.3.2. Artırım Planı

Projenin geliştirme süreci aşağıdaki artırımlara bölünmüştür:

Artırım 1 (Hafta 1-4): Temel Altyapı

- ✓ Core katmanı (Entity'ler, Enum'lar)
- ✓ Data katmanı (Repository, UnitOfWork)
- ✓ Veritabanı migration'ları
- ✓ Temel CRUD işlemleri

Artırım 2 (Hafta 5-6): İş Mantığı

- ✓ Business katmanı (Service'ler)
- ✓ DTO'lar ve Mapping konfigürasyonları
- ✓ Validation (doğrulama) kuralları
- ✓ E-posta servisi altyapısı

Artırım 3 (Hafta 7-10): Kullanıcı Arayüzü

- ✓ Login/Register (Giriş/Kayıt) formları
- ✓ Dashboard ve KPI göstergeleri
- ✓ Proje, Görev ve Takım modülleri

- ✓ Kanban board tasarımı
- ✓ Raporlama ekranları

Artırım 4 (Hafta 11-12): Entegrasyonlar

- ✓ GitHub API entegrasyonu
- ✓ Token pool yönetimi
- ✓ Commit-Task eşleştirme algoritması
- ✓ Web API (Plesk)
- ✓ Davet kabul web arayüzü

Artırım 5 (Hafta 13-14): Test ve Dokümantasyon

- ✓ 177 adet unit test
- ✓ UML diyagramlarının hazırlanması
- ✓ Ekran görüntülerinin alınması
- ✓ Proje sonuç raporunun yazılması

3.3.3. GANTT İş Akış Diyagramı

Projenin geliştirme sürecini ve fazların tamamlanma durumunu gösteren iş akış çizelgesi aşağıdadır.

Proje Geliştirme Takvimi

Faz	Görev Tanımı	Başlangıç	Bitiş	Süre	Durum
Faz 1	Login & Auth	Hafta 1	Hafta 1	2 gün	Tamamlandı
Faz 2	Dashboard Layout	Hafta 1	Hafta 1	4 gün	Tamamlandı
Faz 3	Projects Content	Hafta 2	Hafta 2	5 gün	Tamamlandı
Faz 4	Tasks Content	Hafta 2-3	Hafta 3	6 gün	Tamamlandı
Faz 5	Team Management	Hafta 3-4	Hafta 4	5 gün	Tamamlandı
Faz 6	Reports & Analytics	Hafta 4-5	Hafta 5	5 gün	Tamamlandı
Faz 7	GitHub Integration	Hafta 5-6	Hafta 6	6 gün	Tamamlandı
Faz 8	Web API & Invitations	Hafta 6-7	Hafta 7	4 gün	Tamamlandı
Faz 9	Testing & Refinement	Hafta 7	Hafta 7	4 gün	Tamamlandı
Faz 10	Documentation	Hafta 8	Hafta 8	5 gün	Tamamlandı

Proje Geliştirme Süreci (GANTT Şeması)

Faz / Görev	Hafta 1	Hafta 2	Hafta 3	Hafta 4	Hafta 5	Hafta 6	Hafta 7	Hafta 8
Faz 1: Login								
Faz 2: Dash								
Faz 3: Proje								
Faz 4: Görev								
Faz 5: Takım								
Faz 6: Rapor								
Faz 7: GitHub								

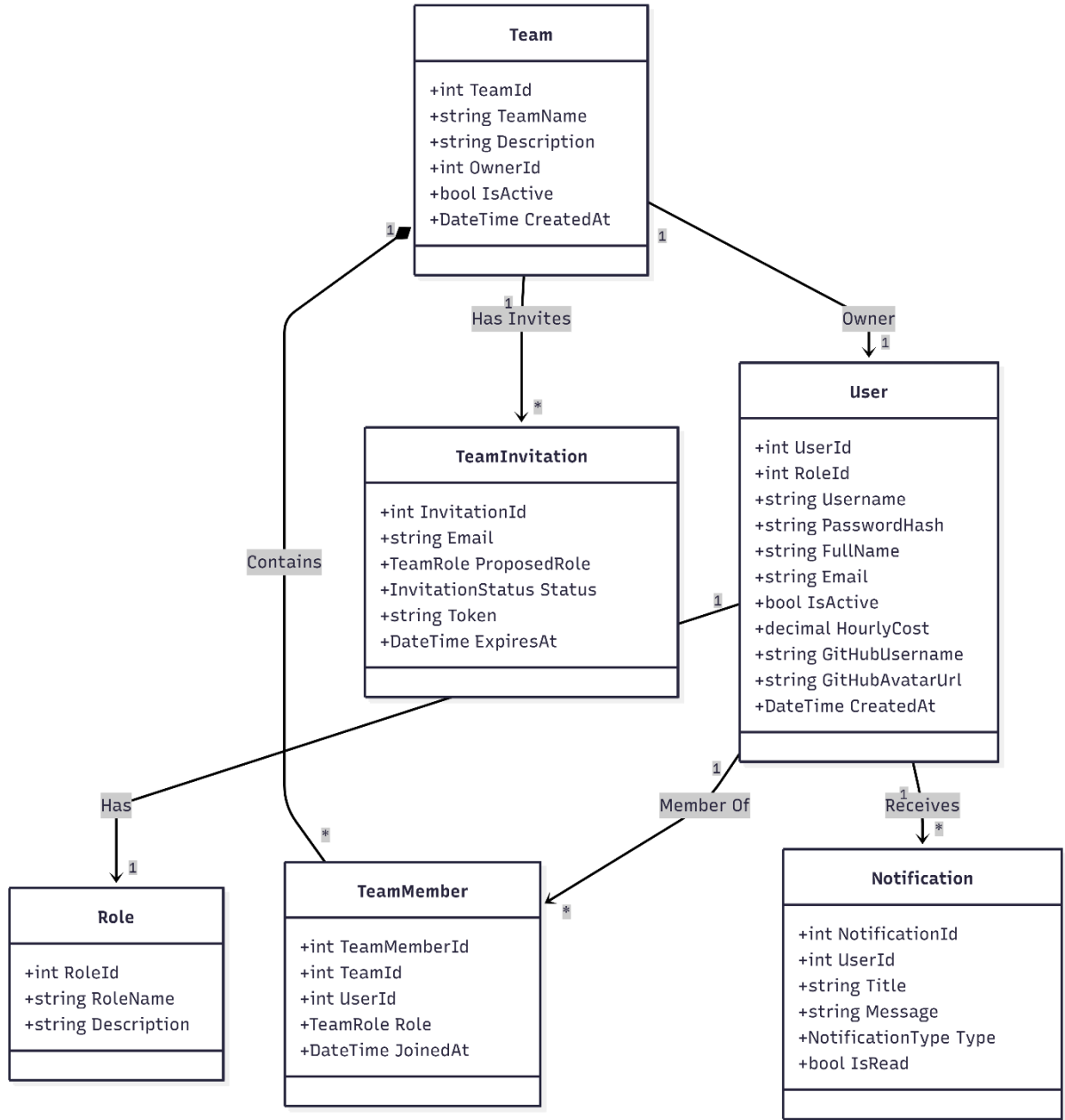
Faz 8: WebAPI									
Faz 9: Test									
Faz 10: Dok.									

(Not: Yukarıdaki tabloda boyalı alanlar, ilgili fazın aktif olarak yürütüldüğü zaman aralığını temsil etmektedir.)

3.4. Class Diyagramları

3.4.1. Kimlik ve Organizasyon Katmanı (Identity & Core Layer)

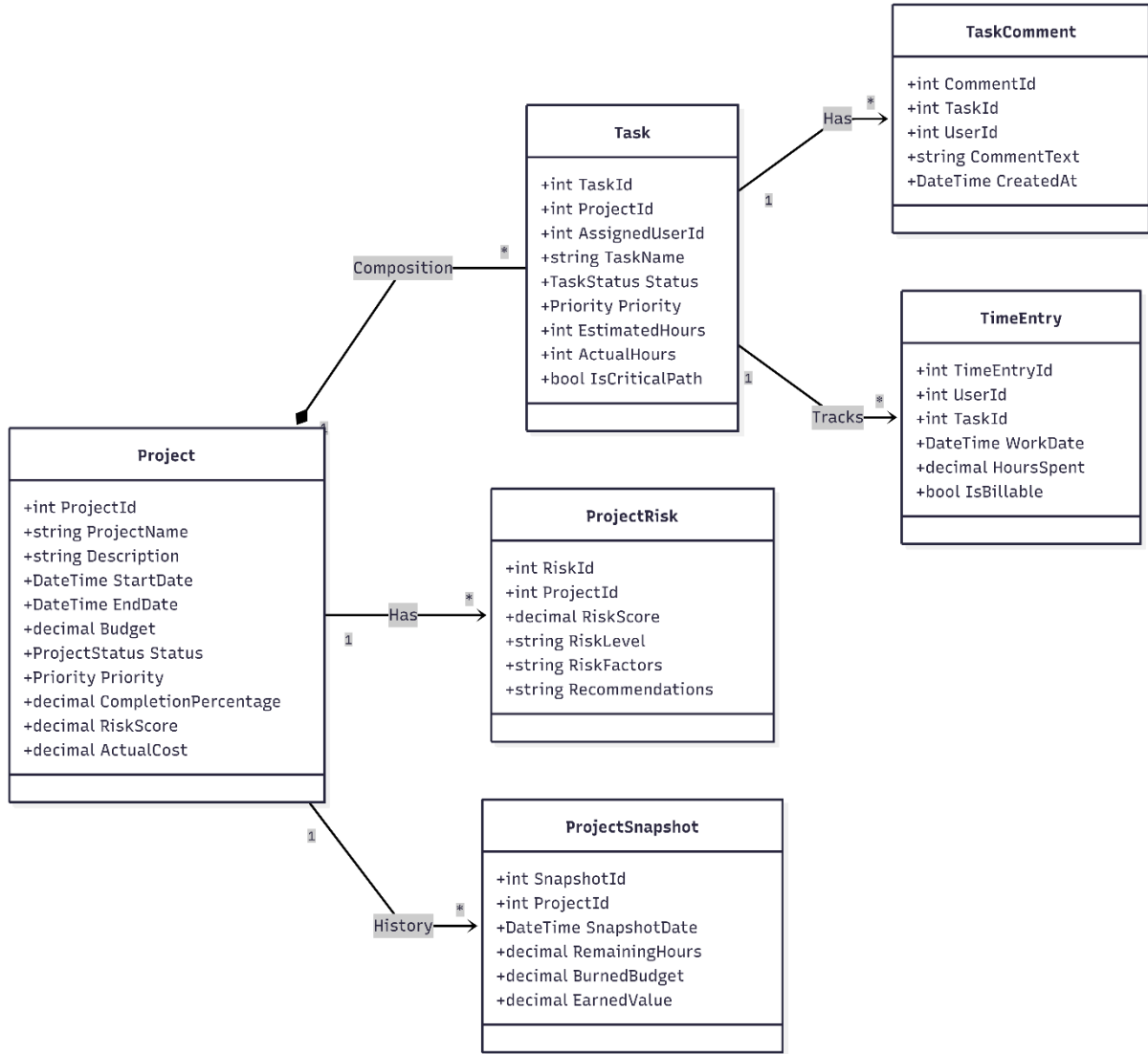
Sistemin omurgasını oluşturan bu katman; kullanıcı yönetimi, rol tabanlı yetkilendirme (RBAC) ve takım organizasyonunu yönetir. Şekil 3.5'te görüldüğü üzere, User (Kullanıcı) sınıfı merkezi bir konumdadır. Role sınıfı ile N:1 (Çoka-Bir), Team sınıfı ile ise hem sahiplik hem de üyelik üzerinden karmaşık ilişkiler kurar.



3.4.2. Proje Yönetim Katmanı (Project Management Layer)

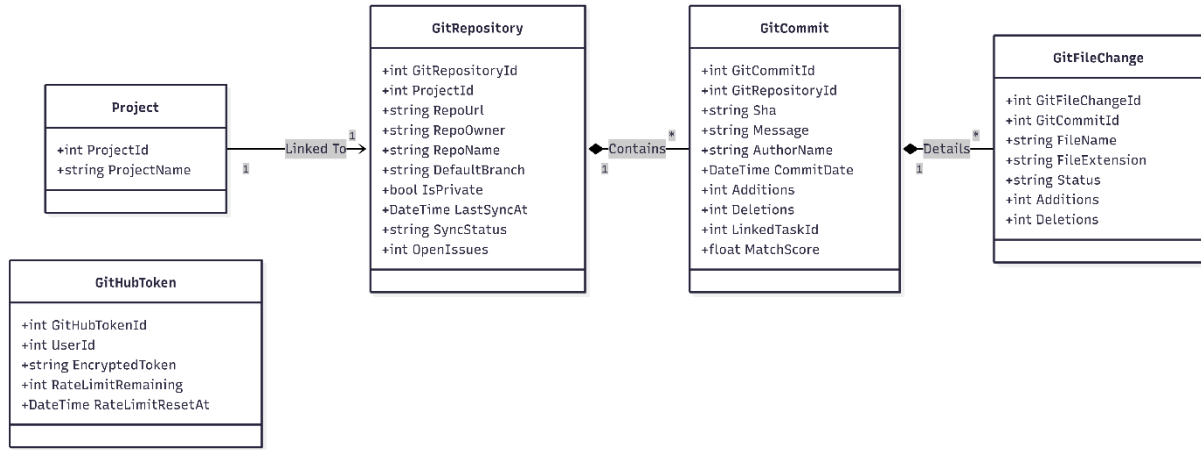
Uygulamanın ana iş mantığını (Business Logic) barındıran katmandır. Project sınıfı, Task (Görev) sınıflarını kapsayan (Composition) ana varlıktır. ProjectRisk sınıfı projenin risk analiz

verilerini, TimeEntry sınıfı ise görevler üzerindeki efor takibini yönetir. Bu yapı, projenin planlamadan tamamlanmaya kadar olan yaşam döngüsünü modeller.



3.4.3. Git Entegrasyon Katmanı (Git Integration Layer)

Proje yönetim süreçlerini teknik kodlama süreçleriyle birleştiren katmandır. GitRepository sınıfı, bir projeye 1:1 ilişki ile bağlanır. Şekil 3.7'deki hiyerarşide görüleceği üzere; GitCommit ve GitFileChange sınıfları, GitHub üzerinden çekilen verileri hiyerarşik bir yapıda tutar. GitCommit sınıfı içerisindeki LinkedTaskId alanı, yazılım kodlarını yönetimsel görevlerle (Tasks) ilişkilendirerek izlenebilirlik (Traceability) sağlar.

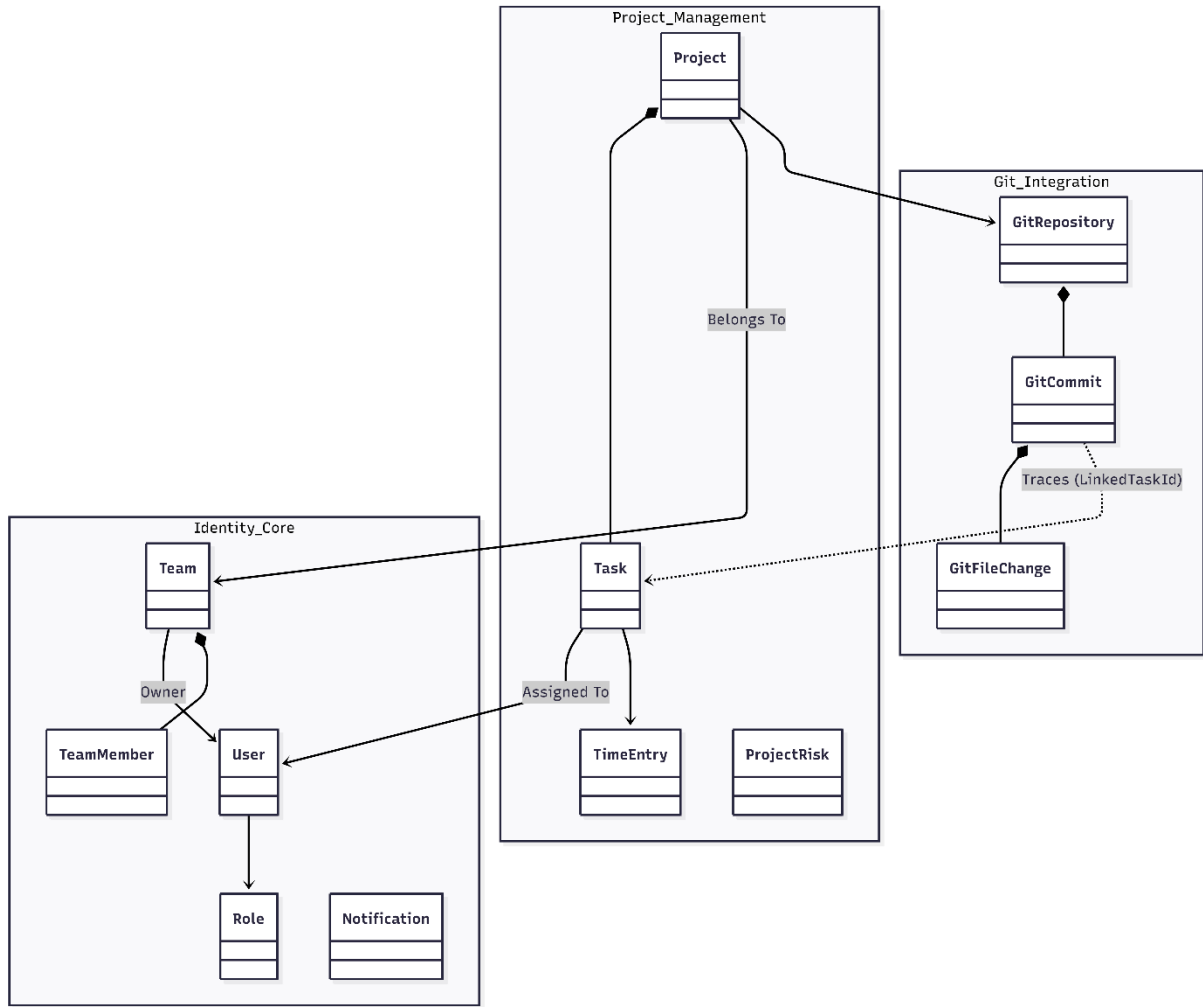


3.4.4. Genel Sistem Mimarisi ve Modüller Arası İlişkiler (Tam Görünüm)

Önceki bölümlerde detaylandırılan üç ana modülün (Identity, Project Management, Git Integration) birbiriyle nasıl entegre çalıştığını gösteren kuş bakışı (high-level) sınıf diyagramı Şekil 3.8'de sunulmuştur.

Bu diyagram, sistemin bütünlüğünü sağlayan şu kritik bağlantıları görselleştirir:

- İş Atama: User (Identity) ile Task (PM) arasındaki atama ilişkisi.
- Sahiplik: Project (PM) ile Team (Identity) arasındaki aidiyet ilişkisi.
- İzlenebilirlik: GitCommit (Git) ile Task (PM) arasındaki kod-görev izlenebilirlik ilişkisi (LinkedTaskId).



Kritik Enum Tanımları

Sınıf diyagramlarında kullanılan ve sistemin durum yönetimini (State Management)sağlayan kritik numaralandırma (Enum) tipleri aşağıdadır:

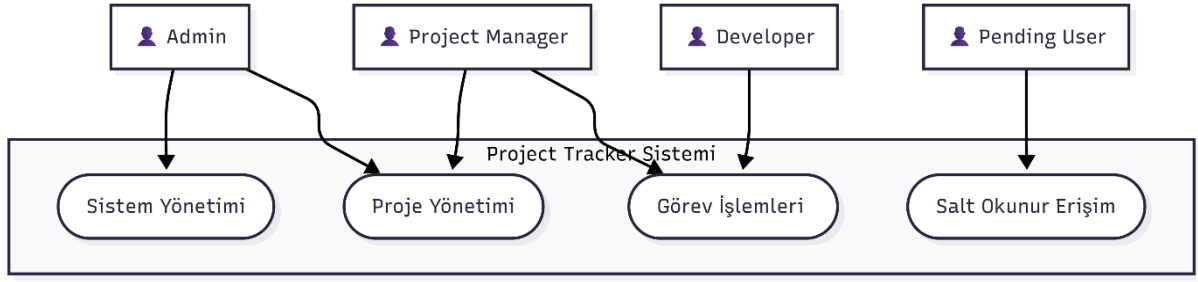
- **ProjectStatus:** Projenin yaşam döngüsünü (Planlandı, Aktif, Tamamlandı) belirler.
- **Priority:** Görevlerin öncelik sırasını (Düşük, Orta, Kritik) yönetir.
- **TeamRole:** Takım içerisindeki hiyerarşiyi (Sahip, Yönetici, Geliştirici) tanımlar.
- **NotificationType:** Kullanıcıya gösterilecek bildirim türünü (Bilgi, Hata, Başarı) ayırır.

3.5. Use Case Diyagramı

3.5.1. Use Case Diyagramı (Kullanım Senaryoları)

Project Tracker sistemindeki kullanıcı etkileşimlerini ve yetki sınırlarını modellemek için UML Use Case diyagramları kullanılmıştır. Sistemde tanımlı 4 temel aktör bulunmaktadır: Admin, ProjectManager, Developer ve Pending (Onay Bekleyen).

Şekil'te, bu aktörlerin sistem üzerindeki genel yetki ve sorumluluk alanlarını gösteren üst seviye (High-Level) diyagram yer almaktadır.



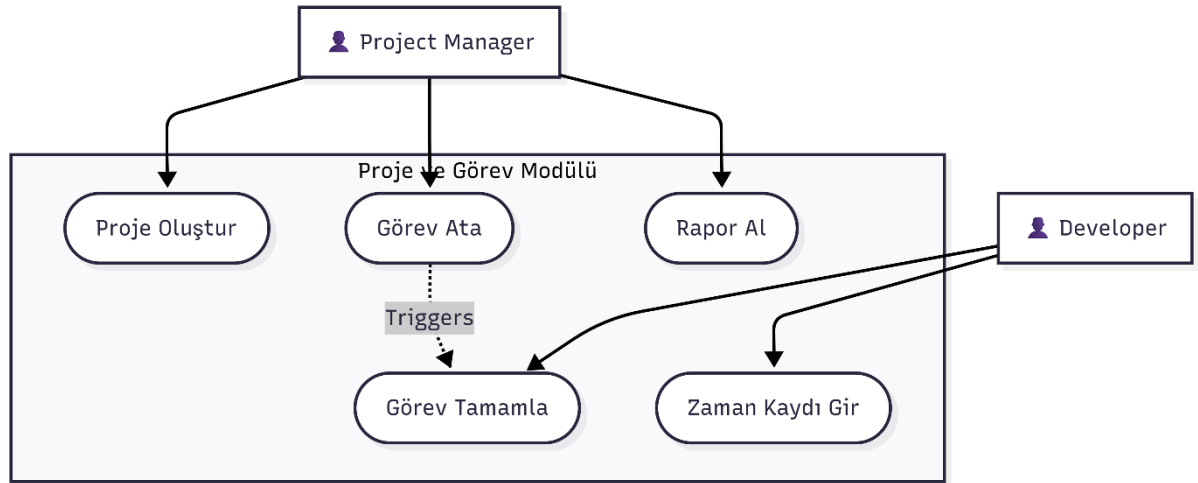
Şekil 3.5.1: Genel Aktör ve Yetki Diyagramı

3.5.2. Modül Bazlı Detaylı Senaryolar

Sistemin karmaşıklığı nedeniyle, kullanım senaryoları mantıksal modüllere ayrılarak detaylandırılmıştır.

A. Proje ve Görev Yönetimi Senaryoları

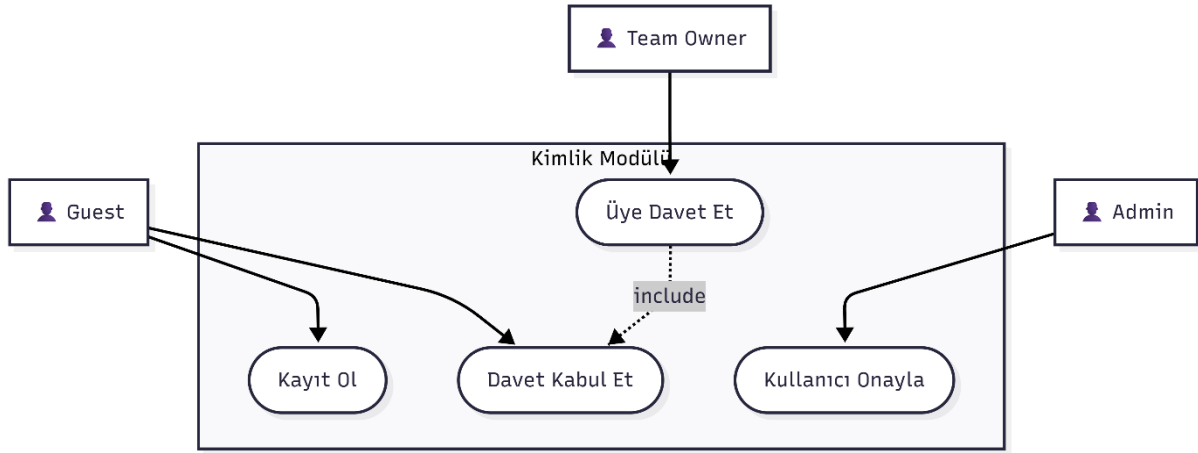
Proje yöneticileri ve geliştiricilerin günlük iş akışlarını kapsar. Proje oluşturma, görev atama, durum güncelleme ve zaman takibi gibi kritik işlemler bu diyagramda modellenmiştir.



Şekil 3.5.2: Proje ve Görev Yönetimi Use Case Diyagramı

B. Takım ve Kimlik Yönetimi Senaryoları

Sisteme yeni üye alımı, davet süreçleri ve takım organizasyonunu içeren idari süreçlerdir.



Şekil 3.5.3: Takım ve Kimlik Yönetimi Use Case Diyagramı

3.5.3. Use Case Senaryo Listesi

Aşağıdaki tabloda, sistemdeki kritik kullanım senaryolarının (Use Cases) aktörleri ve ön koşulları özetlenmiştir.

Tablo 3.1: Kritik Kullanım Senaryoları

No	Senaryo Adı	Birincil Aktör	Açıklama
UC-01	Kayıt Ol & Giriş Yap	Misafir	Kullanıcı hesabı oluşturma ve oturum açma işlemleri.
UC-06	Proje Oluştur	ProjectManager	Yeni bir proje başlatma ve parametrelerini belirleme.
UC-12	Görev Ata	ProjectManager	Mevcut bir görevi takım üyesine zimmetleme.
UC-14	Durum Güncelle	Developer	Görevi "Pending" durumundan "Completed" durumuna çekme.
UC-21	Üye Davet Et	Admin/Owner	E-posta yoluyla takıma dışarıdan üye çağırma.
UC-30	GitHub Token Ekle	Developer	Kişisel erişim belirtecini sisteme tanımlama.
UC-37	Audit Log İzle	Admin	Sistemdeki tüm hareketleri ve güvenlik loglarını inceleme.

3.6. Arayüz Tasarımı - Modüllerin ve Formların Tanıtımı

Uygulamayı geliştirirken kullanıcı deneyimini ön planda tuttum. DevExpress kontrolleri sayesinde profesyonel görünümlü, modern bir arayüz elde ettim. Dark theme tercih ettim çünkü göz yorgunluğunu azaltıyor ve modern bir görünüm sağlıyor.

3.6.1. Giriş Modülü

Giriş

Ekranı

(FrmLogin)

PROJECT TRACKER
Manage Analyze Your Projects Easily
500+ Projects Managed
YMH 219 - Object Oriented Programming Project
v1.1.0
by @bilalabic

SIGN IN

Username:
Enter username...

Password:

LOGIN

CANCEL

[Don't have an account? Create one](#)

Bu ekranda kullanıcılar sisteme giriş yapıyor. Sol tarafta projenin logosu ve tanıtımı, sağ tarafta ise giriş formu var. Kullanıcı adı ve şifre girildikten sonra BCrypt ile şifre doğrulaması yapılıyor. Eğer kullanıcı "Pending" rolündeyse bekleme ekranına, değilse ana dashboard'a yönlendiriliyor.

Kayıt Ekranı (FrmRegister)

PROJECT TRACKER
Manage Analyze Your Projects Easily
500+ Projects Managed
YMH 219 - Object Oriented Programming Project
v1.1.0
by @bilalabic

SIGN UP

Username:
Enter username...

Full Name:
Enter your full name...

Email:
Enter email address...

Password:
Enter password...

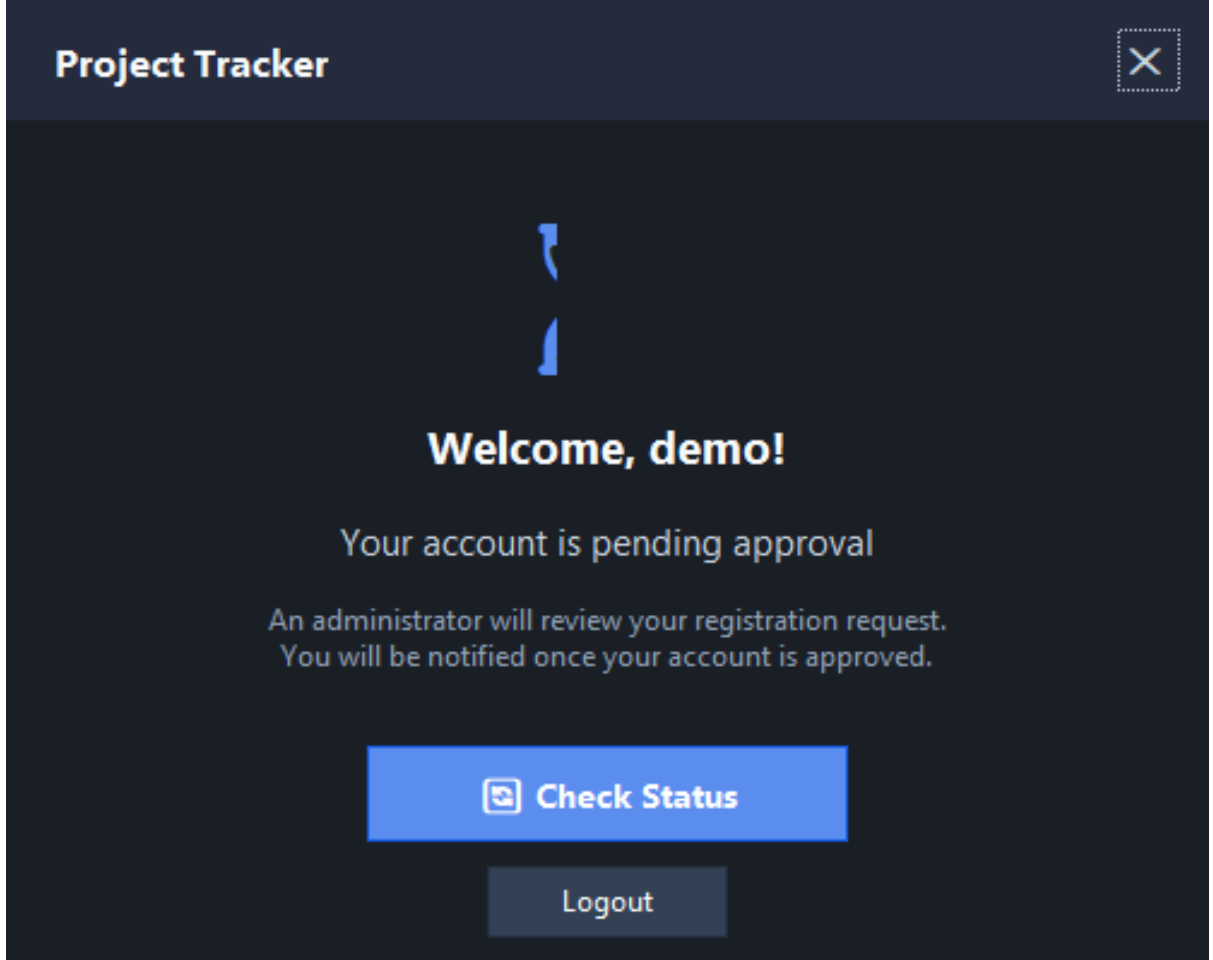
Confirm Password:
Confirm password...

REGISTER

[← Back to Login](#)

Yeni kullanıcılar bu ekrandan kayıt oluyor. Kullanıcı adı, tam ad, e-posta ve şifre alanları var. FluentValidation ile tüm girişler doğrulanıyor. Eğer kullanıcı bir davet linki ile geldiyse, invitation token otomatik olarak algılanıyor ve davetteki rol atanıyor.

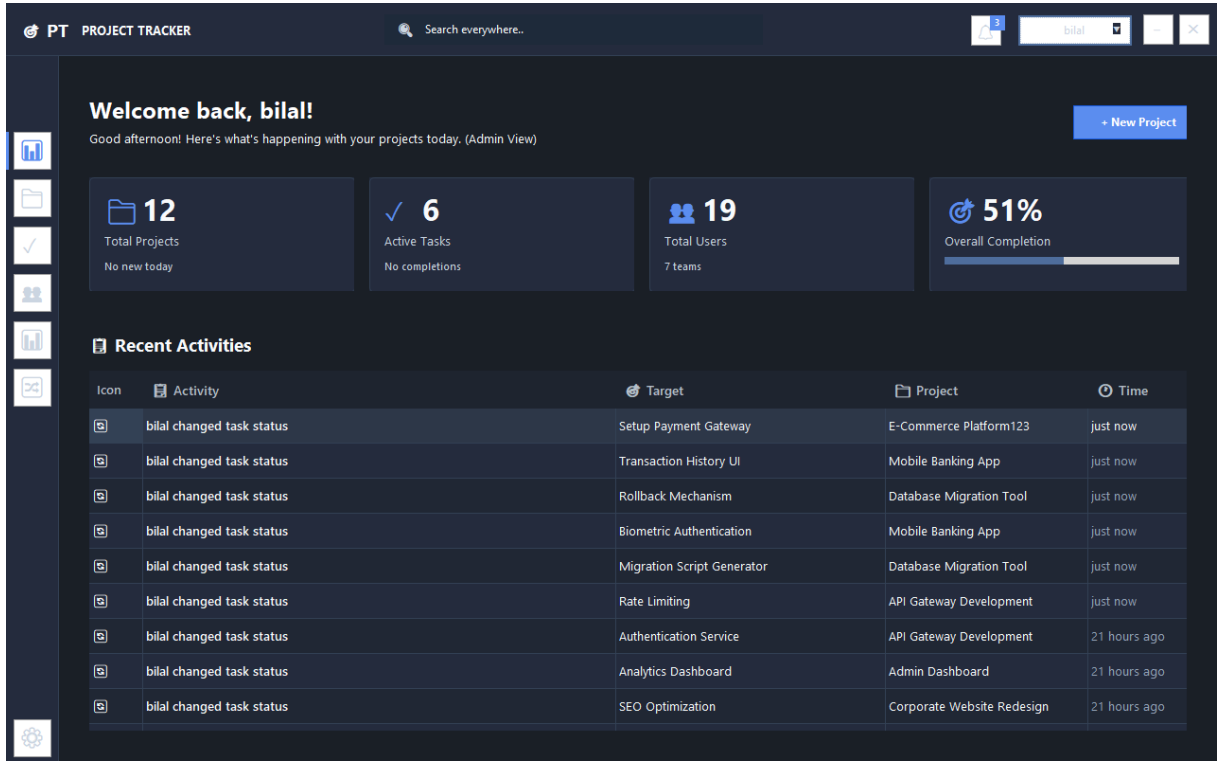
Onay Bekleme Ekranı (FrmPendingWaitlist)



Direkt kayıt olan kullanıcılar bu ekranı görüyor. "Admin onayınız bekleniyor" mesajı gösteriliyor. Admin onayladıktan sonra kullanıcı sisteme giriş yapabilir hale geliyor.

3.6.2. Dashboard Modülü

Ana Form (FrmDashboard) - Master Container



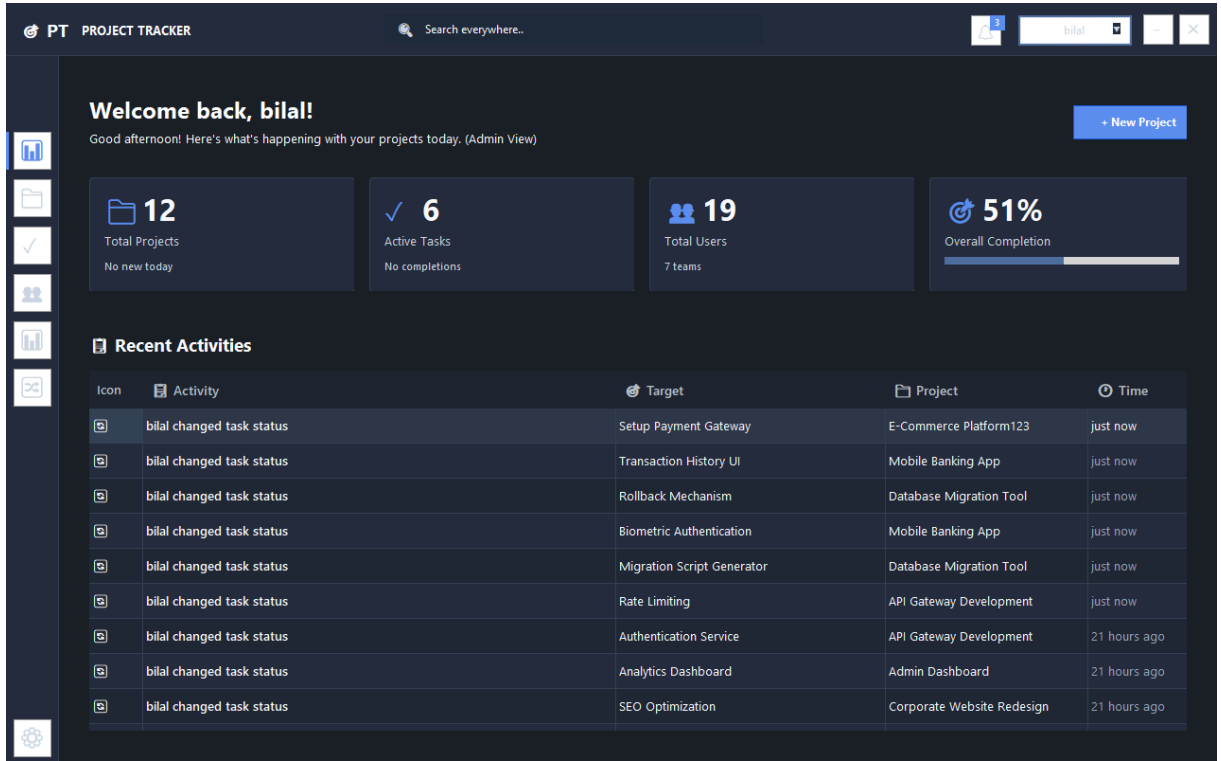
FrmDashboard, uygulamanın ana çatısını oluşturan master form. Giriş yapıldıktan sonra açılan bu form, tüm içerik panellerini (Content) barındıran bir container görevi görüyor. Yapısı şöyle:

- **Sol Sidebar:** Navigasyon menüsü - tüm modüllere erişim butonları
- **Üst Bar:** Kullanıcı bilgisi, bildirimler, çıkış butonu
- **Ana Panel:** Seçilen content'in yüklendiği alan

Sidebar'dan bir menü öğesine tıklandığında, ilgili UserControl (örn: ProjectsContent, TasksContent) ana panele dinamik olarak yükleniyor. Bu yaklaşım sayesinde:

- Tek bir form üzerinden tüm modüllere erişim sağlanıyor
- Kod tekrarı önleniyor
- Bellek kullanımı optimize ediliyor (sadece aktif content yüklü)

Dashboard İçeriği (DashboardContent)



Burası uygulamanın kalbi! FrmDashboard açıldığında varsayılan olarak yüklenen content. Üst kısımda KPI kartları var:

- Toplam proje sayısı
- Aktif görev sayısı
- Takım sayısı
- Bekleyen görevler

Alt kısımda ise son aktiviteler listesi (audit log'dan çekiliyor) ve hızlı erişim butonları var.

3.6.3. Proje Modülü

Proje Listesi (ProjectsContent)

- Başlangıç ve bitiş tarihi
- Bütçe
- Öncelik seviyesi (Low, Medium, High, Critical)
- Atanacak takım
- GitHub repository URL (opsiyonel)

Proje Düzenleme

PT PROJECT TRACKER

Edit Project
Editing: E-Commerce Platform123

Project Name *
E-Commerce Platform123

Description
Building a modern e-commerce platform with React and Next.js

Start Date * 10/29/2025 **End Date** 4/29/2026

Status Active **Priority** High

Team * Frontend Development **Budget** 150000.00

Project Tasks
8 tasks (0 done, 1 in progress)

- Design Product Catalog UI**
Blocked High
Sarah Johnson 29 Nov
- Implement Shopping Cart**
Blocked Critical
Emma Davis 03 Jan
- Setup Payment Gateway**
InProgress High
David Brown 13 Jan
- Performance Testing**
Blocked Medium
Unassigned 18 Jan
- Cart Item Addition**
Completed High
David Brown 22 Dec

Buttons: Cancel, Save Project

Mevcut projeleri düzenlerken ek olarak şunları görebilirsiniz:

- Tamamlanma yüzdesi (görevlere göre otomatik hesaplanıyor)
- Risk skoru (0-100 arası, algoritma ile hesaplanıyor)
- Proje durumu değiştirme
- Projeye ait görevlerin listesi

3.6.4. Görev Modülü

Görev Listesi - Grid Görünümü

PT PROJECT TRACKER

Search everywhere..

3 bilal

Tasks

Manage tasks and track progress

Kanban View + New Task

All Projects All Priority Clear

Task Name	Project	Assignee	Status	Priority	Due Date	Progress	Actions
Design Product Catalog UI	E-Commerce Platform123	Sarah Johnson	Blocked	High	29 Nov 2025	0%	
Implement Shopping Cart	E-Commerce Platform123	Emma Davis	Blocked	Critical	03 Jan 2026	0%	
Setup Payment Gateway	E-Commerce Platform123	David Brown	InProgress	High	13 Jan 2026	50%	
Performance Testing	E-Commerce Platform123		Blocked	Medium	18 Jan 2026	0%	
Homepage Redesign	Corporate Website Red...	Emma Davis	Blocked	High	01 Dec 2025	0%	
SEO Optimization	Corporate Website Red...	Emma Davis	Blocked	High	05 Jan 2026	0%	
User Management Module	Admin Dashboard	David Brown	Completed	Critical	25 Dec 2025	100%	
Analytics Dashboard	Admin Dashboard	David Brown	Blocked	High	15 Jan 2026	0%	
Authentication Service	API Gateway Development	Alex Martinez	Blocked	Critical	25 Nov 2025	0%	
Rate Limiting	API Gateway Development	Sophia Wilson	InProgress	High	30 Dec 2025	50%	
API Documentation	API Gateway Development	Alex Martinez	Completed	Medium	10 Jan 2026	100%	
Migration Script Generator	Database Migration Tool	Sophia Wilson	Pending	High	15 Jan 2026	0%	

Showing 27 of 27 tasks

Refresh

Klasik liste görünümü. Proje ve kullanıcıya göre filtreleme yapılabilir. Durum ve öncelik kolonları renkli gösteriliyor. Çift tıklama ile görev detayına gidiliyor.

Görev Listesi - Kanban Görünümü

PT PROJECT TRACKER

Search everywhere..

3 bilal

Tasks

Manage tasks and track progress

List View + New Task

All Projects All Priority Clear

TO DO	IN PROGRESS	COMPLETED	BLOCKED
Migration Script Generator Database Migration Tool High • 15 Jan	Setup Payment Gateway E-Commerce Platform123 High • 13 Jan	User Management Module Admin Dashboard Critical • 25 Dec	Design Product Catalog UI E-Commerce Platform123 High • 29 Nov
Rollback Mechanism Database Migration Tool Critical • 05 Feb	Rate Limiting API Gateway Development High • 30 Dec	API Documentation API Gateway Development Medium • 10 Jan	Implement Shopping Cart E-Commerce Platform123 Critical • 03 Jan
Transaction History UI Mobile Banking App High • 10 Jan	Biometric Authentication Mobile Banking App Critical • 08 Jan	Push Notifications Mobile Banking App Medium • 25 Jan	Performance Testing E-Commerce Platform123 Medium • 18 Jan
		Requirements Gathering Fitness Tracker App High • 05 Jan	Homepage Redesign Corporate Website Redesign High • 01 Dec

Showing 27 of 27 tasks

Refresh

Bu benim en sevdiğim özelliklerden biri! 4 kolon var:

- Pending (Bekleyen) - Gri
- InProgress (Devam Eden) - Mavi

- Completed (Tamamlanan) - Yeşil
- Blocked (Engellenen) - Kırmızı

Görevleri sürükle-bırak ile kolonlar arasında taşıyabilirsiniz. Durum otomatik güncelleniyor ve audit log'a kaydediliyor.

Görev Düzenleme

The screenshot shows the 'Edit Task' interface in the 'PT PROJECT TRACKER' application. The form is titled 'Edit Task' and includes a 'Back' button. The fields are as follows:

- Task Name ***: Change artifact upload path for GitHub Pages
- Description**: Change artifact upload path for GitHub Pages
- Project ***: Project Tacker
- Assignee**: elmas gümüş
- Start Date ***: 1/6/2026
- Due Date**: 1/30/2026
- Status**: Pending
- Priority**: High

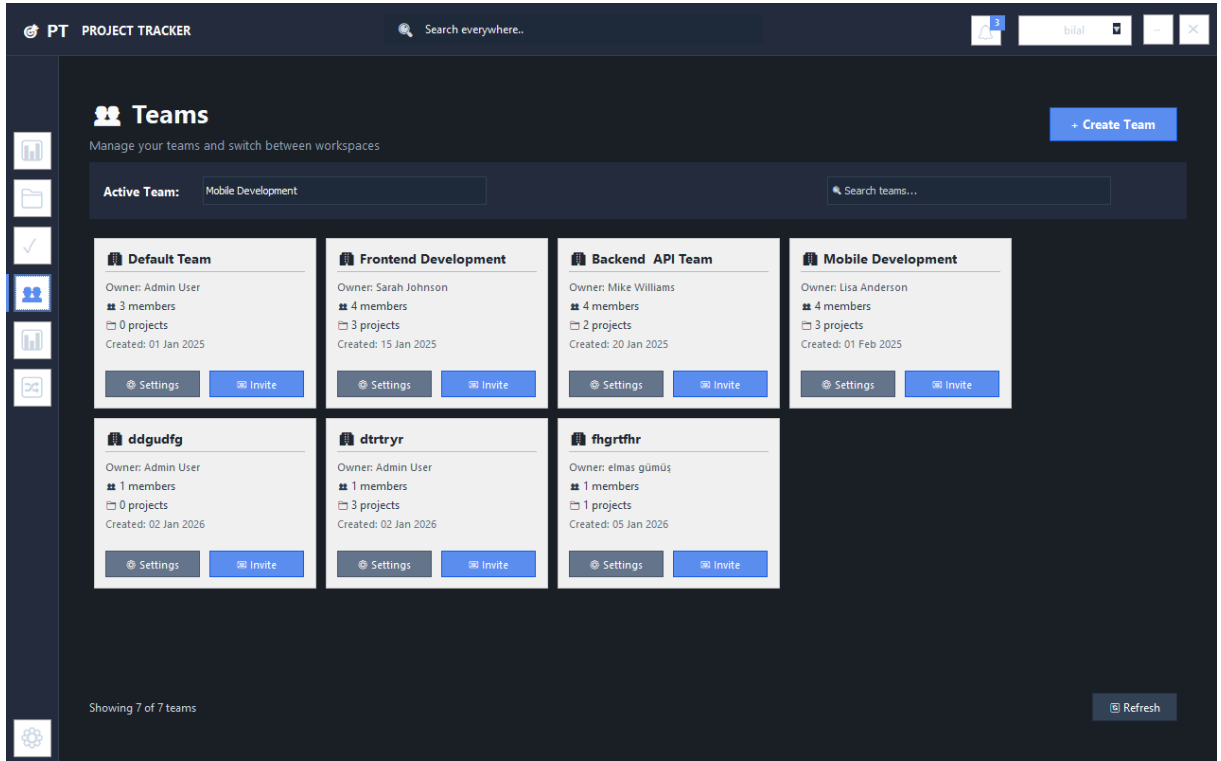
At the bottom, there are 'Cancel' and 'Update Task' buttons. On the right, the 'Related Commits' section shows a commit [291857d] titled 'Change artifact upload path for GitHub P...' by BILAL ABİÇ, 20h ago, with a change of +43 lines. A summary at the bottom right states 'Total: 1 commit: +43 / -0 lines'.

Görev detaylarını düzenlerken:

- Görev adı ve açıklaması
- Hangi projeye ait
- Kime atandığı (atama yapıldığında e-posta gidiyor)
- Öncelik ve durum
- Tahmini ve gerçek çalışma saati
- Başlangıç ve bitiş tarihi
- Eşleşen GitHub commit'leri (varsa)

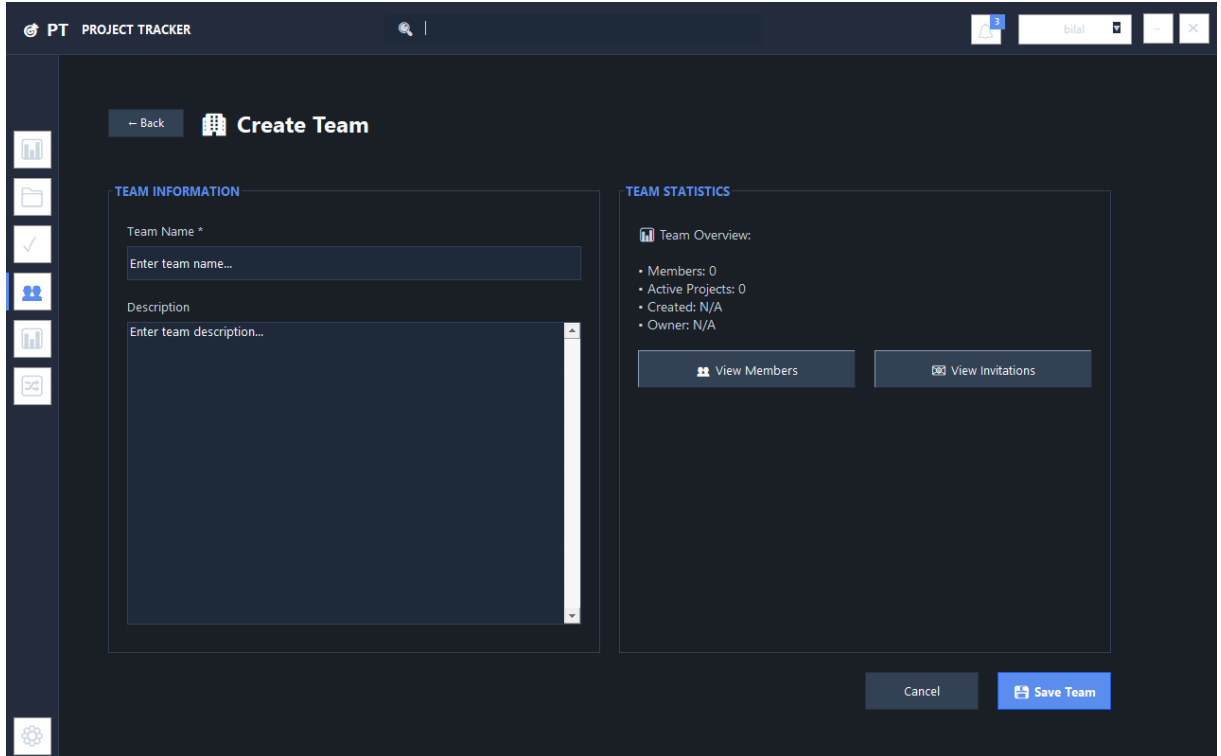
3.6.5. Takım Modülü

Takım Listesi (TeamsContent)



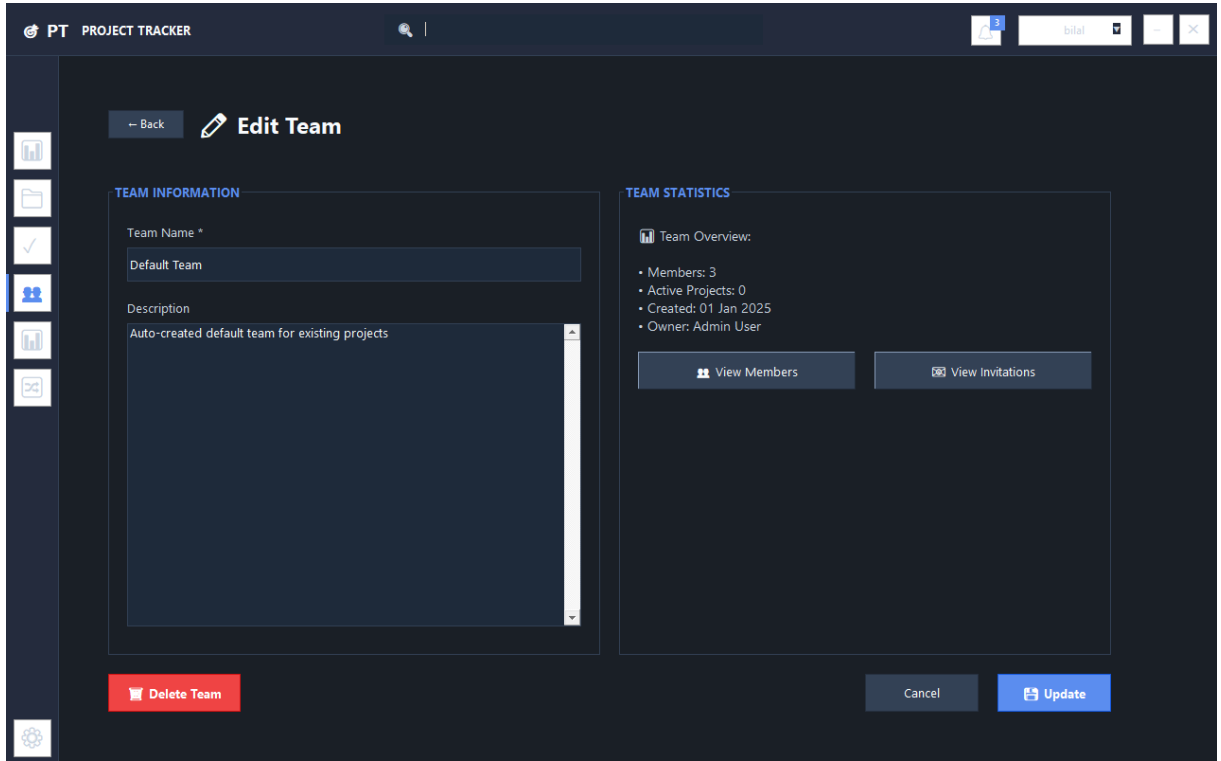
Kullanıcının üye olduğu tüm takımlar listeleniyor. Sahip olduğu takımlar özel bir işaretle belirtiliyor. Takım kartlarında üye sayısı ve proje sayısı gösteriliyor.

Takım Oluşturma



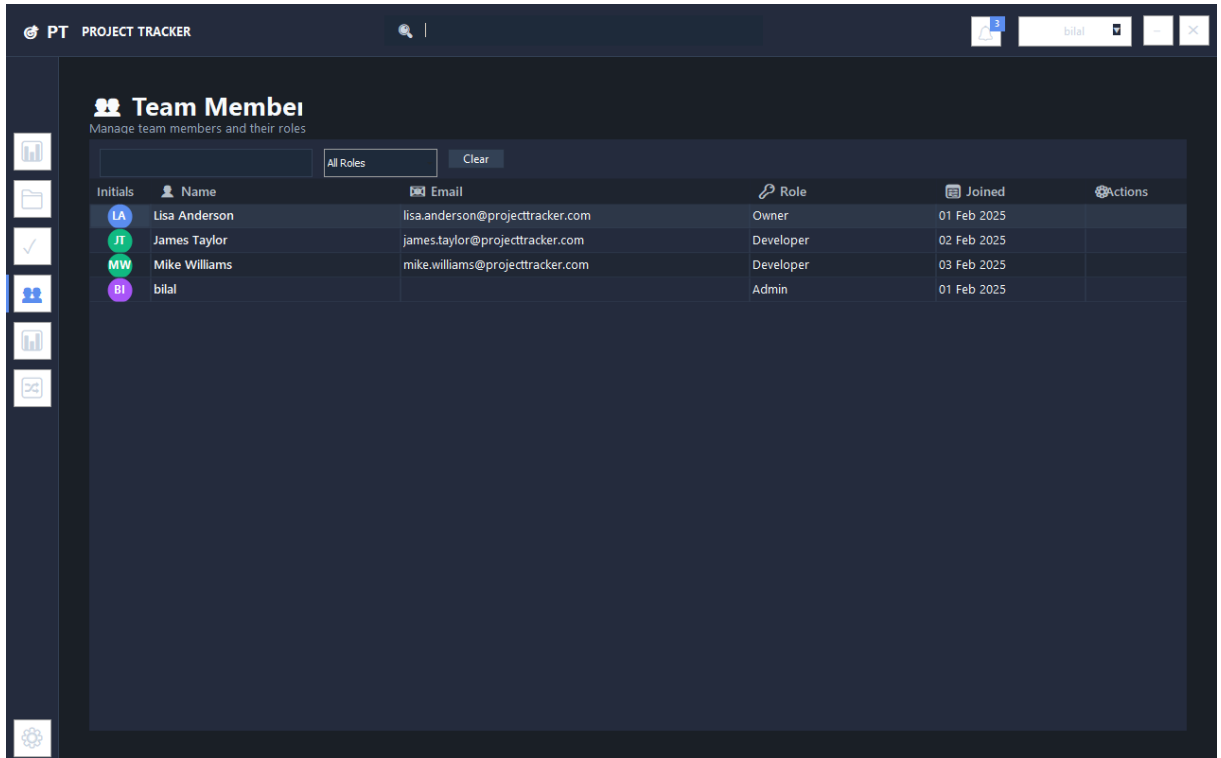
Yeni takım oluştururken sadece ad ve açıklama yeterli. Oluşturan kişi otomatik olarak "Owner" rolüyle ekleniyor.

Takım Düzenleme



Takım bilgilerini düzenleme, üye listesini görme ve yeni üye davet etme işlemleri bu ekrandan yapılıyor.

Takım Üyeleri (TeamMembersContent)

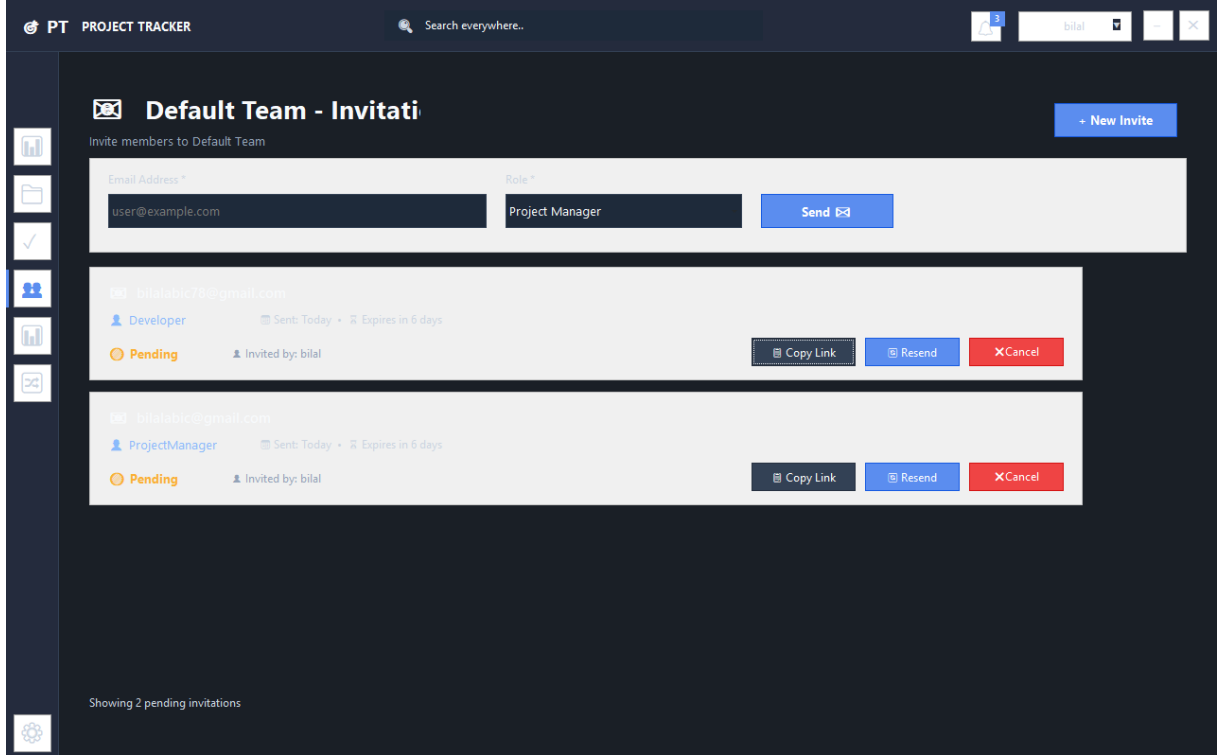


Takımdaki tüm üyeler ve rolleri (Owner, Admin, Developer) listeleniyor. Admin yetkisi olanlar:

- Üye rolünü değiştirebilir

- Üyeyi takımdan çıkarabilir
- Pending kullanıcıları onaylayabilir

Takım Davetleri (InvitationsContent)



PT PROJECT TRACKER Search everywhere...

Default Team - Invitations

Invite members to Default Team

+ New Invite

Email Address * Role *

user@example.com Project Manager Send

totalabir78@gmail.com

Developer Sent: Today Expires in 6 days

Pending Invited by: bilal

Copy Link Resend Cancel

totalabir@gmail.com

ProjectManager Sent: Today Expires in 6 days

Pending Invited by: bilal

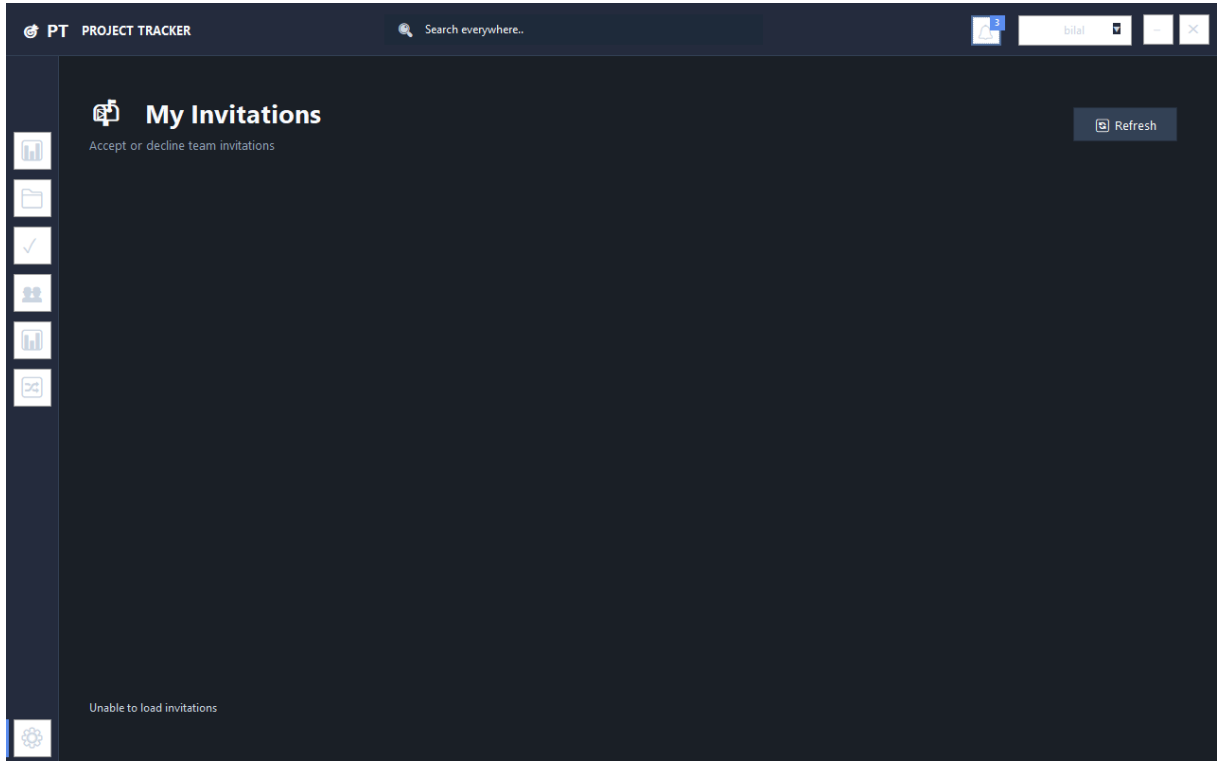
Copy Link Resend Cancel

Showing 2 pending invitations

Gönderilen davetlerin listesi. Her davetin durumu görünüyor:

- Pending: Henüz yanıt verilmedi (Sarı)
- Accepted: Kabul edildi (Yeşil)
- Declined: Reddedildi (Kırmızı)
- Expired: Süresi doldu (Gri)

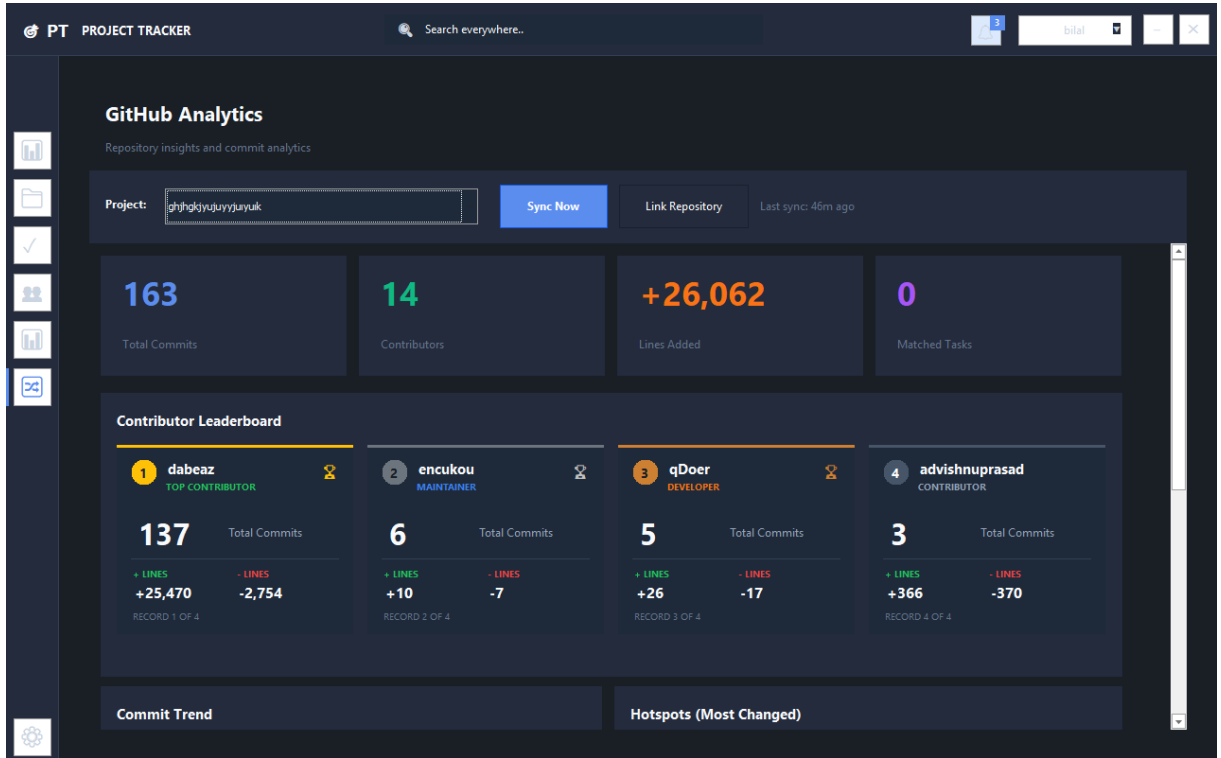
Gelen Davetlerim (MyInvitationsContent)



Kullanıcıya gelen davetler burada listeleniyor. Kabul veya Red butonlarıyla yanıt verilebiliyor.

3.6.6. GitHub Modülü

GitHub Analytics - Commit Listesi



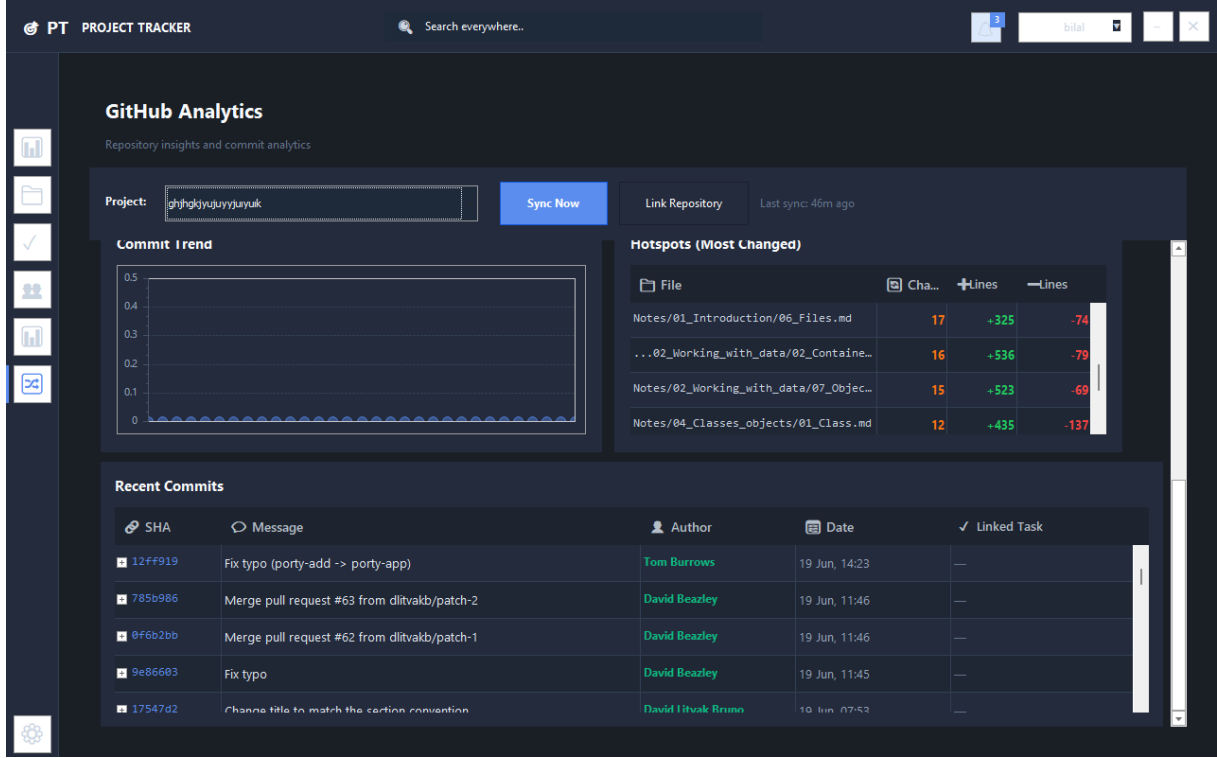
Projeye bağlı GitHub repository'den çekilen commit'ler burada listeleniyor. Her commit için:

- SHA (kısa)
- Commit mesajı

- Yazar ve avatar
- Tarih
- Eklenen/silinen satır sayısı
- Eşleşen görev (varsa)

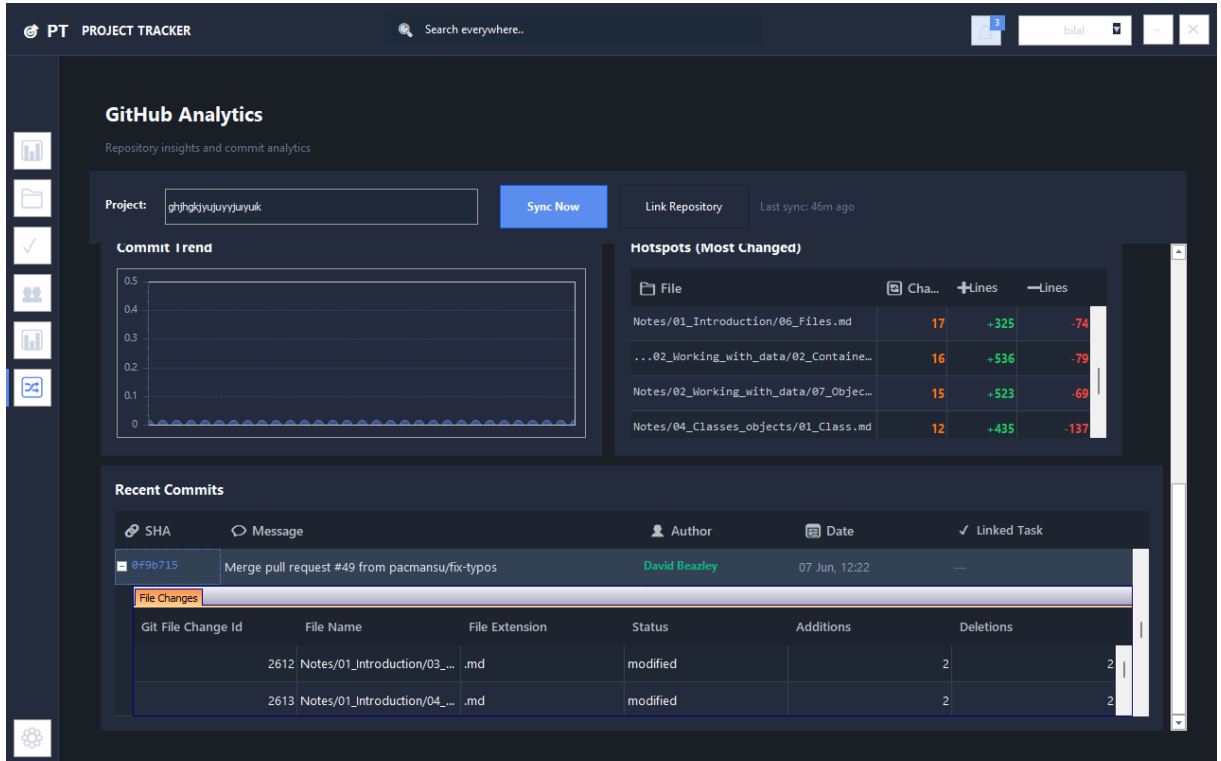
"Sync" butonuyla en son commit'ler çekilebiliyor.

GitHub Analytics - Contributor İstatistikleri



Pie chart ile her geliştiricinin commit sayısı görselleştiriliyor. En aktif geliştiriciler kolayca görülebiliyor. Ayrıca toplam commit, branch ve contributor sayıları KPI kartlarında gösteriliyor.

GitHub Analytics - File Hotspots



En çok deęişiklik yapılan dosyalar listeleniyor. Bu özellik, kod tabanındaki "hotspot"ları (sık deęişen, potansiyel olarak sorunlu alanları) tespit etmeye yardımcı oluyor.

3.6.7. Raporlama Modülü

Proje Bazlı Rapor

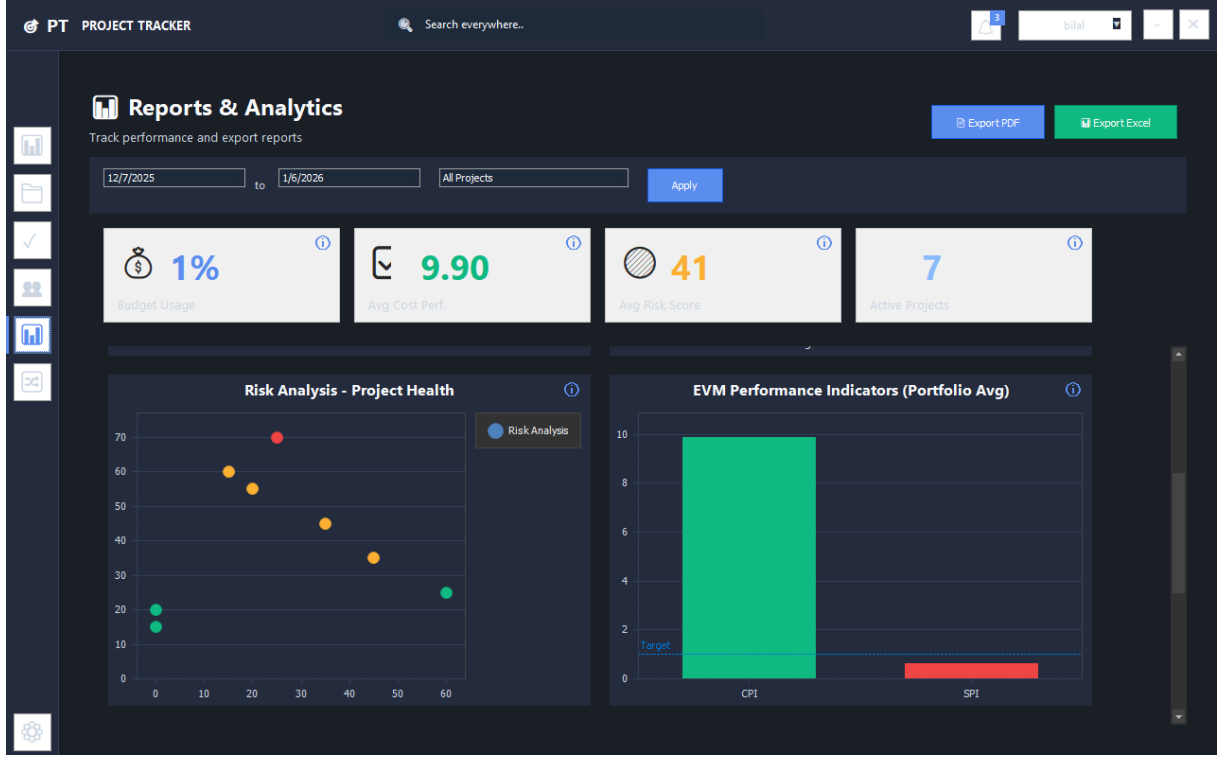


Seçilen projenin detaylı raporu:

- Görev dağılımı (durum bazlı pie chart)

- Tamamlanma yüzdesi
- Risk skoru ve risk faktörleri
- Takım üyelerinin performansı

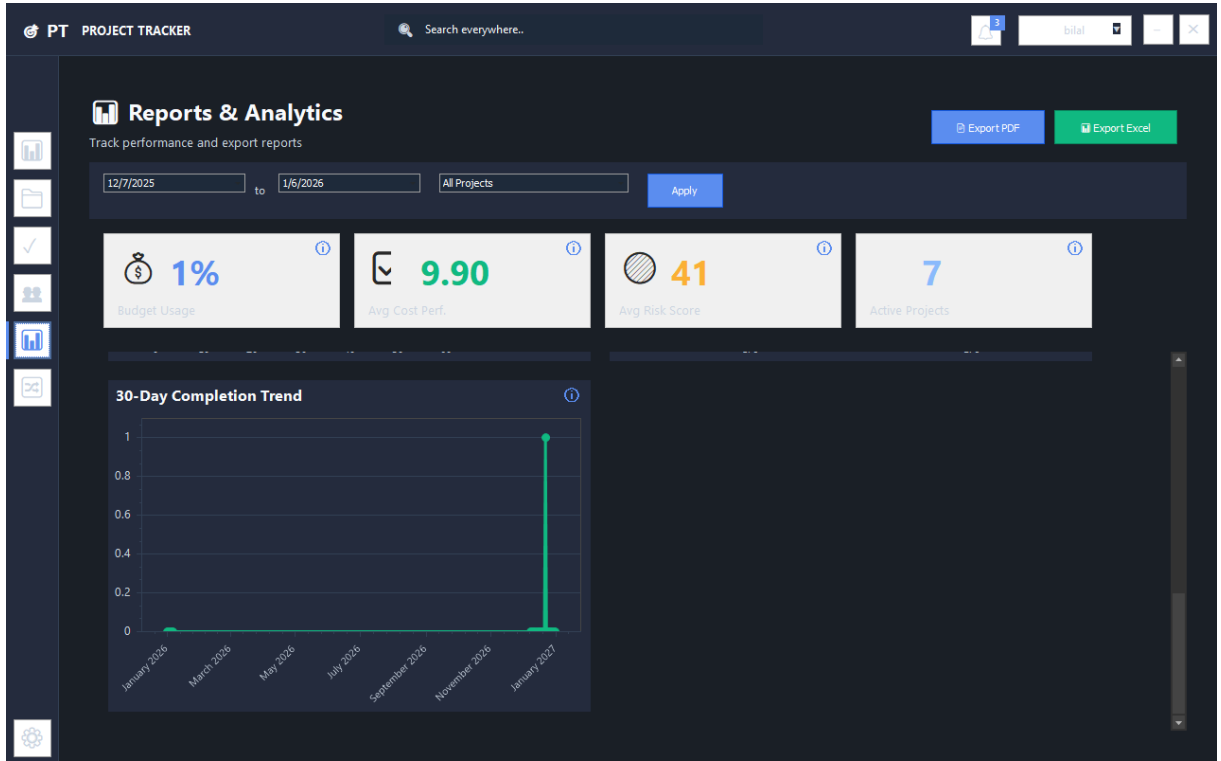
Kullanıcı Bazlı Rapor



Seçilen kullanıcının performans raporu:

- Atanan görev sayısı
- Tamamlanan görev sayısı
- Ortalama tamamlama süresi
- Verimlilik metrikleri

Takım Bazlı Rapor



Takım performans raporu:

- Üye bazlı istatistikler
- Proje ilerlemeleri
- Toplam çalışma saatleri

Tüm raporlar PDF ve Excel formatında export edilebiliyor.

3.6.8. Ayarlar Modülü

Kullanıcı Ayarları (UserSettingsContent)

PT PROJECT TRACKER Search everywhere..

User Settings

Manage your profile and GitHub integration settings

Profile Information

Full Name:

Email:

Department:

GitHub Integration

GitHub Username:

GitHub Token:

Token Status: Not configured

Kullanıcı kendi profilini düzenleyebiliyor:

- Ad, e-posta güncelleme
- Şifre değiştirme
- GitHub kullanıcı adı bağlama
- Profil fotoğrafı (GitHub'dan çekiliyor)

3.6.9. Hata Yönetimi

Özel Mesaj Kutusu (FrmMessage)

PROJECT TRACKER

Manage Analyze Your Projects Easily

500+ Projects Managed

YMH 219 - Object Oriented Programming Project

v1.1.0 by @bilalabic

SIGN IN

Username:

Don't have an account? Create one

Error

Invalid username or password

Bir hata oluştuğunda veya bilgi verilmesi gerektiğinde kullanıcı dostu mesajlar gösteriliyor. Dark-themed özel mesaj kutusu tasarladım. Mesaj türüne göre farklı renkler kullanılıyor:

- Başarı: Yeşil
- Bilgi: Mavi
- Uyarı: Sarı
- Hata: Kırmızı

4. KODLAMA

4.1. Programlama Dili - Neden C#?

4.1.1. C# Seçim Nedenleri

Neden	Açıklama
Ders Gerksinimleri	Nesne Tabanlı Programlama dersi .NET ve C# odaklı
Windows Forms Desteği	Native Windows UI geliştirme
Modern Syntax	C# 12.0 ile primary constructors, pattern matching
Güçlü Tip Sistemi	Compile-time hata yakalama
LINQ Desteği	Veritabanı sorguları için güçlü araç
Async/Await	Asenkron programlama kolaylığı
Entity Framework	ORM desteği
Geniş Ekosistem	NuGet paketleri, topluluk desteği

4.1.2. C# 12.0 Özellikleri Kullanımı

```
// Primary Constructor (C# 12)
public class UserService(IUnitOfWork unitOfWork, IMapper mapper)
{
    private readonly IUnitOfWork _unitOfWork = unitOfWork;
    private readonly IMapper _mapper = mapper;
}

// Pattern Matching
var result = status switch
{
    TaskStatus.Pending => "Bekliyor",
    TaskStatus.InProgress => "Devam Ediyor",
    TaskStatus.Completed => "Tamamlandı",
    _ => "Bilinmiyor"
};

// Null-conditional Operators
var userName = user?.FullName ?? "Anonim";

// Collection Expressions (C# 12)
List<string> roles = ["Admin", "ProjectManager", "Developer", "Pending"];
```

4.2. Modüller

4.2.1. Core Modülü (ProjectTracker.Core)

Amaç: Domain entity'leri ve interface'leri barındırır.

İçerik:

- 18 Entity sınıfı

- 7 Enum tanımı
- Repository interface'leri
- IUnitOfWork interface'i

Örnek Entity: User.cs

```
public class User
{
    public int UserId { get; set; }
    public int RoleId { get; set; }
    public string Username { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public bool IsActive { get; set; } = true;
    public string? GitHubUsername { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.Now;

    // Navigation Properties
    public virtual Role Role { get; set; } = null!;
    public virtual ICollection<Project> CreatedProjects { get; set; } = new
List<Project>();
    public virtual ICollection<Task> AssignedTasks { get; set; } = new
List<Task>();
}
```

4.2.2. Data Modülü (ProjectTracker.Data)

Amaç: Veritabanı erişim katmanı.

İçerik:

- AppDbContext (EF Core DbContext)
- Generic Repository<T>
- UnitOfWork
- Migrations

```
// Generic Repository Pattern
public class Repository<T> : IRepository<T> where T : class
{
    protected readonly AppDbContext _context;
    protected readonly DbSet<T> _dbSet;

    public Repository(AppDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public async Task<T?> GetByIdAsync(int id)
        => await _dbSet.FindAsync(id);

    public async Task<IEnumerable<T>> GetAllAsync()
        => await _dbSet.ToListAsync();

    public async Task AddAsync(T entity)
        => await _dbSet.AddAsync(entity);
}
```

```

    public void Update(T entity)
    => _dbSet.Update(entity);

    public void Delete(T entity)
    => _dbSet.Remove(entity);
}

```

4.2.3. Business Modülü (ProjectTracker.Business)

Amaç: İş mantığı ve servisler.

İçerik:

- 14 Service sınıfı
- 27+ DTO sınıfı
- Validator sınıfları
- AutoMapper profilleri

Service Listesi:

Service	Açıklama
UserService	Kullanıcı CRUD, login, rol yönetimi
ProjectService	Proje CRUD, risk hesaplama
TaskService	Görev CRUD, durum değişikliği
TeamService	Takım CRUD, üye yönetimi
InvitationService	Davet gönderme, kabul/red
AuditLogService	Aktivite loglama
ReportService	Temel raporlar
AdvancedReportService	Gelişmiş analitik
EmailService	SMTP e-posta gönderimi
RemoteInvitationService	Plesk API entegrasyonu
TokenPoolService	GitHub token havuzu yönetimi
TaskMatchingService	Commit-Task eşleştirme
GitHubSyncService	Repository senkronizasyonu
GitHubAnalyticsService	GitHub istatistikleri

Örnek Service: ProjectService.cs

```

public class ProjectService : IProjectService
{
    private readonly IUnitOfWork _unitOfWork;
    private readonly IMapper _mapper;
    private readonly IAuditLogService _auditLogService;

    public async Task<ProjectDto> CreateAsync(CreateProjectDto dto, int
userId)
    {
        var project = _mapper.Map<Project>(dto);
        project.CreatedByUserId = userId;
        project.CreatedAt = DateTime.Now;

        await _unitOfWork.Projects.AddAsync(project);
        await _unitOfWork.SaveChangesAsync();

        await _auditLogService.LogAsync("Projects", project.ProjectId,

```

```

        "Created", null, project, userId);

    return _mapper.Map<ProjectDto>(project);
}
}

```

4.2.4. UI Modülü (ProjectTracker.UI)

Amaç: Windows Forms kullanıcı arayüzü.

İçerik:

- 3 Ana Form (Login, Register, Dashboard)
- 12 UserControl (Content/Detail)
- Helper sınıfları
- DI Container yapılandırması

Form/Control Listesi:

Form/Control	Açıklama
FrmLogin	Giriş ekranı
FrmRegister	Kayıt ekranı
FrmPendingWaitlist	Onay bekleme ekranı
FrmDashboard	Ana dashboard (sidebar + content)
DashboardContent	KPI kartları, grafikler
ProjectsContent	Proje listesi
ProjectDetailControl	Proje detay/düzenleme
TasksContent	Görev listesi + Kanban
TaskDetailControl	Görev detay/düzenleme
TeamsContent	Takım listesi
TeamDetailControl	Takım detay/düzenleme
TeamMembersContent	Takım üyeleri
InvitationsContent	Davetler
MyInvitationsContent	Gelen davetlerim
GitHubContent	GitHub Analytics
ReportsContent	Raporlar
UserSettingsContent	Kullanıcı ayarları

4.2.5. API Modülü (ProjectTracker.API)

Amaç: Web üzerinden davet kabul için REST API.

İçerik:

- Minimal API endpoints
- InvitationDbContext
- CORS yapılandırması

```

// Minimal API Endpoints
app.MapGet("/api/invitations/validate", async (string token,
    InvitationDbContext db) =>
{
    var invitation = await db.Invitations
        .FirstOrDefaultAsync(i => i.Token == token);

    if (invitation == null)

```

```

        return Results.NotFound(new { isValid = false });

        return Results.Ok(new {
            isValid = true,
            teamName = invitation.TeamName,
            invitedBy = invitation.InvitedByName,
            proposedRole = invitation.ProposedRole
        });
    });

app.MapPost("/api/invitations/accept", async (AcceptRequest request,
InvitationDbContext db) =>
{
    var invitation = await db.Invitations
        .FirstOrDefaultAsync(i => i.Token == request.Token);

    if (invitation == null)
        return Results.NotFound();

    invitation.Status = "Accepted";
    invitation.RespondedAt = DateTime.Now;
    await db.SaveChangesAsync();

    return Results.Ok(new { success = true });
});

```

4.3. Kod Stilleri

4.3.1. Naming Conventions

Öge	Kural	Örnek
Class	PascalCase	`UserService`, `ProjectDto`
Interface	I + PascalCase	`IUserService`, `IRepository`
Method	PascalCase	`GetByIdAsync`, `CreateProject`
Property	PascalCase	`UserId`, `ProjectName`
Private Field	_camelCase	`_unitOfWork`, `_mapper`
Parameter	camelCase	`userId`, `projectDto`
Constant	UPPER_CASE	`MAX_RETRY_COUNT`
Async Method	...Async	`GetAllAsync`, `SaveChangesAsync`

4.3.2. Kod Organizasyonu

```

// Standart sınıf yapısı
public class UserService : IUserService
{
    // 1. Private fields
    private readonly IUnitOfWork _unitOfWork;
    private readonly IMapper _mapper;

    // 2. Constructor
    public UserService(IUnitOfWork unitOfWork, IMapper mapper)
    {
        _unitOfWork = unitOfWork;
        _mapper = mapper;
    }
}

```

```
// 3. Public methods
public async Task<UserDto?> GetByIdAsync(int id)
{
    var user = await _unitOfWork.Users.GetByIdAsync(id);
    return _mapper.Map<UserDto>(user);
}

// 4. Private helper methods
private bool ValidatePassword(string password, string hash)
{
    return BCrypt.Net.BCrypt.Verify(password, hash);
}
}
```

4.3.3. SOLID Prensipleri Uygulaması

Prensip	Uygulaması
Single Responsibility	Her service tek bir entity'den sorumlu
Open/Closed	Generic Repository ile genişletilebilir
Liskov Substitution	Interface'ler ile soyutlama
Interface Segregation	Küçük, odaklı interface'ler
Dependency Inversion	Constructor injection ile DI

4.4. Program Karmaşıklığı

4.4.1. İşlev Nokta Analizi (Function Point Analysis)

Parametre	Toplam
Kullanıcı Girdi (EI)	90
Kullanıcı Çıktı (EO)	134
Kullanıcı Sorgu (EQ)	84
Dahili Mantıksal Dosya (ILF)	212
Harici Arayüz Dosya (EIF)	31
Ayarlanmamış İşlev Noktası (AİN)	551

4.4.2. Teknik Karmaşıklık Faktörü (TKF)

Soru	Puan (0-5)
Veri iletişimi	4
Dağıtık veri işleme	3
Performans	3
Yoğun kullanılan konfigürasyon	3
İşlem hızı	3
Online veri girişi	5
Son kullanıcı verimliliği	4
Online güncelleme	5
Karmaşık işleme	4
Yeniden kullanılabilirlik	4
Kurulum kolaylığı	3
İşletim kolaylığı	4
Çoklu site	2
Değişiklik kolaylığı	4
Toplam TKP	51

$$TKF = 0.65 + (0.01 * 51) = 1.16$$

4.4.3. Ayarlanmış İşlev Noktası

$$\dot{IN} = A\dot{IN} \times TKF$$

$$\dot{IN} = 551 \times 1.16$$

$$\dot{IN} = 639$$

4.4.4. Kod Satır Sayısı Tahmini

Dil	Satır/ $\dot{IN}$	Kullanım Oranı	Tahmini Satır
C#	30	%85	16,294
JavaScript	25	%10	1,598
SQL	15	%5	479
TOPLAM	-	-	~18,371

4.4.5. Karmaşıklık Değerlendirmesi

$\dot{IN}$ Değeri: 639

Karmaşıklık Skalası:

- 0-100: Basit
- 100-300: Orta
- 300-600: Karmaşık
- 600+: Çok Karmaşık

Sonuç: ÇOK KARMAŞIK PROJE

4.4.6. Karmaşıklığı Artıran Faktörler

- **Dual-Database Mimarisi** - Local SQL Server + Plesk Remote DB
- **GitHub API Entegrasyonu** - Rate limiting, token pool yönetimi
- **Akıllı Algoritmalar** - Risk hesaplama, commit-task eşleştirme
- **E-posta Bildirimleri** - SMTP entegrasyonu
- **Katmanlı Mimari** - 5 katmanlı enterprise mimari
- **DevExpress UI** - Profesyonel UI bileşenleri
- **Kanban Board** - Drag & drop görev yönetimi
- **Raporlama Sistemi** - Çoklu rapor türleri

4.4.7. Analiz Sonuçlarının Değerlendirilmesi(Google Gemini AI)

Yapılan İşlev Nokta Analizi sonucunda elde edilen 639 puanlık 'Ayarlanmış İşlev Noktası' değeri, projenin literatürdeki karmaşıklık skalasında "Çok Karmaşık" kategorisinde yer aldığını göstermektedir. Bu veri, projenin sadece temel veri giriş-çıkış işlemlerini içeren bir uygulama olmadığını; GitHub entegrasyonu, çift veritabanı mimarisi ve arka plan algoritmaları nedeniyle yüksek bir teknik yoğunluğa sahip olduğunu ortaya koymaktadır. Hesaplanan yaklaşık 18.000 satırlık kod hacmi ve 1.16 olarak belirlenen Teknik Karmaşıklık Faktörü (TKF), sistem tasarımında kullanılan 5 katmanlı mimarinin ve geniş kapsamlı veritabanı yapısının, projenin sürdürülebilirliği ve ölçeklenebilirliği açısından gerekliliğini desteklemektedir.

4.5. Akıllı Algoritmalar

4.5.1. Risk Skoru Hesaplama Algoritması

Amaç: Projelerin gecikme riskini 0-100 arasında hesaplamak.

Formül:

$$\text{RiskSkoru} = (\text{GörevSayısı} \times 0.3) + ((100 - \text{TamamlanmaOranı}) \times 0.4) + ((1 / \text{TakımBüyükülüğü}) \times 0.2) + (\text{BütçeKullanımOranı} \times 0.3)$$

Sonuç Değerlendirmesi:

Skor Aralığı	Risk Seviyesi	Renk
0-40	Düşük Risk	Yeşil
41-70	Orta Risk	Sarı
71-100	Yüksek Risk	Kırmızı

4.5.2. Commit-Task Eşleştirme Algoritması

Amaç: GitHub commit'lerini otomatik olarak ilgili görevlere bağlamak.

Formül:

$$\text{EşleşmeSkoru} = (\text{TaskAdıBenzerliği} \times 0.4) + (\text{AnahtarKelimeEşleşmesi} \times 0.3) + (\text{TaskIDEşleşmesi} \times 0.3)$$

Algoritma Adımları:

- I. Task ID Pattern Arama (#123, TASK-123, [123])
- II. Kelime Benzerliği Hesaplama
- III. Levenshtein Distance ile benzerlik
- IV. Skor > 0.3 ise eşleşme kabul

Örnek:

Task: "Login Bug Fix"

Commit: "Fixed login validation bug #42"

TaskIDEşleşmesi: 0.0 (ID eşleşmedi)

AnahtarKelimeEşleşmesi: 0.67 ("login", "bug", "fix" kelimelerinden 2'si eşleşti)

TaskAdıBenzerliği: 0.45 (Levenshtein distance)

Toplam Skor: $(0.45 \times 0.4) + (0.67 \times 0.3) + (0.0 \times 0.3) = 0.38$ Eşleşme olumlu!

5. DOĞRULAMA VE GEÇERLEME

5.1. Arayüz Çalışıyor mu? (Fonksiyonel Testler)

Her modül için arayüz testleri yapıldı. Aşağıda test edilen senaryolar ve sonuçları:

5.1.1. Giriş Modülü Testleri

Test Senaryosu	Beklenen Sonuç	Gerçek Sonuç	Durum
Doğru kullanıcı adı/şifre ile giriş	Dashboard açılır	Dashboard açıldı	Başarılı
Yanlış şifre ile giriş	Hata mesajı gösterilir	Hata mesajı gösterildi	Başarılı
Boş kullanıcı adı ile giriş	Validation hatası	Validation hatası gösterildi	Başarılı

Pending kullanıcı girişi	Bekleme ekranı açılır	Bekleme ekranı açıldı	Başarılı
Kayıt formu doldurma	Kullanıcı oluşturulur	Kullanıcı oluşturuldu	Başarılı

5.1.2. Dashboard Modülü Testleri

Test Senaryosu	Beklenen Sonuç	Gerçek Sonuç	Durum
KPI kartları yükleniyor	Doğru sayılar gösterilir	Doğru sayılar gösterildi	✓ Başarılı
Son aktiviteler listesi	Audit log'dan veri çekilir	Veriler çekildi	✓ Başarılı
Sidebar navigasyonu	İlgili content açılır	Content'ler açıldı	✓ Başarılı

5.1.3. Proje Modülü Testleri

Test Senaryosu	Beklenen Sonuç	Gerçek Sonuç	Durum
Proje listesi yükleniyor	Projeler listelenir	Projeler listelendi	✓ Başarılı
Yeni proje oluşturma	Proje kaydedilir	Proje kaydedildi	✓ Başarılı
Proje düzenleme	Değişiklikler kaydedilir	Değişiklikler kaydedildi	✓ Başarılı
Proje silme	Proje silinir	Proje silindi	✓ Başarılı
GitHub repo bağlama	Repo bağlanır	Repo bağlandı	✓ Başarılı

5.1.4. Görev Modülü Testleri

Test Senaryosu	Beklenen Sonuç	Gerçek Sonuç	Durum
Görev listesi (Grid)	Görevler listelenir	Görevler listelendi	✓ Başarılı
Görev listesi (Kanban)	4 kolon gösterilir	4 kolon gösterildi	✓ Başarılı
Kanban sürük-le-bırak	Durum güncellenir	Durum güncellendi	✓ Başarılı
Görev oluşturma	Görev kaydedilir	Görev kaydedildi	✓ Başarılı
Görev atama	E-posta gönderilir	E-posta gönderildi	✓ Başarılı

5.1.5. Takım Modülü Testleri

Test Senaryosu	Beklenen Sonuç	Gerçek Sonuç	Durum
Takım listesi	Takımlar listelenir	Takımlar listelendi	Başarılı
Takım oluşturma	Takım kaydedilir	Takım kaydedildi	Başarılı
Üye ekleme	Üye eklenir	Üye eklendi	Başarılı
Davet gönderme	E-posta gönderilir	E-posta gönderildi	Başarılı
Web'den davet kabul	Davet kabul edilir	Davet kabul edildi	Başarılı

5.1.6. GitHub Modülü Testleri

Test Senaryosu	Beklenen Sonuç	Gerçek Sonuç	Durum
Commit listesi	Commit'ler çekilir	Commit'ler çekildi	Başarılı
Sync butonu	Yeni commit'ler çekilir	Yeni commit'ler çekildi	Başarılı
Contributor istatistikleri	Pie chart gösterilir	Pie chart gösterildi	Başarılı
File hotspots	En çok değişen dosyalar	Dosyalar listelendi	Başarılı

5.1.7. Raporlama Modülü Testleri

Test Senaryosu	Beklenen Sonuç	Gerçek Sonuç	Durum
Proje raporu	Rapor gösterilir	Rapor gösterildi	Başarılı
PDF export	PDF dosyası oluşur	PDF oluşturuldu	Başarılı
Excel export	Excel dosyası oluşur	Excel oluşturuldu	Başarılı

5.2. Unit Test Sonuçları

5.2.1. Test Özeti

Metrik	Değer
Toplam Test Sayısı	177
Başarılı	177
Başarısız	0
Başarı Oranı	%100

5.2.2. Servis Bazlı Test Dağılımı

Servis	Test Sayısı	Durum
UserService	17	Başarılı
ProjectService	12	Başarılı
TaskService	12	Başarılı
TeamService	14	Başarılı
InvitationService	18	Başarılı
AuditLogService	9	Başarılı
ReportService	7	Başarılı
TokenPoolService	10	Başarılı
TaskMatchingService	8	Başarılı
GitHubAnalyticsService	14	Başarılı
GitHubSyncService	14	Başarılı
EmailService	12	Başarılı
AdvancedReportService	18	Başarılı
RemoteInvitationService	12	Başarılı
GENEL TOPLAM	177	%100

5.2.3. Test Teknolojileri

Teknoloji	Versiyon	Kullanım Amacı
xUnit	2.5.3	Test Framework (Ana yapı)
Moq	4.20.70	Mocking Library (Bağımlılık soyutlama)
FluentAssertions	6.12.0	Assertion Library (Doğrulama)
InMemory DB	8.0.0	Test Veritabanı (İzole ortam)

5.2.4. Örnek Test Kodu

```
public class UserServiceTests
{
    private readonly Mock<IUnitOfWork> _mockUnitOfWork;
    private readonly Mock<IMapper> _mockMapper;
    private readonly UserService _userService;

    public UserServiceTests ()
    {
        _mockUnitOfWork = new Mock<IUnitOfWork> ();
        _mockMapper = new Mock<IMapper> ();
        _userService = new UserService (_mockUnitOfWork.Object,
        _mockMapper.Object);
    }

    [Fact]
    public async Task GetByIdAsync_ExistingUser_ReturnsUserDto ()
    {

```

```

// Arrange
var user = new User { UserId = 1, Username = "testuser" };
var userDto = new UserDto { UserId = 1, Username = "testuser" };

_mockUnitOfWork.Setup(u => u.Users.GetByIdAsync(1))
    .ReturnsAsync(user);
_mockMapper.Setup(m => m.Map<UserDto>(user))
    .Returns(userDto);

// Act
var result = await _userService.GetByIdAsync(1);

// Assert
result.Should().NotNull();
result!.UserId.Should().Be(1);
result.Username.Should().Be("testuser");
}

[Fact]
public async Task LoginAsync_ValidCredentials_ReturnsUser()
{
    // Arrange
    var passwordHash = BCrypt.Net.BCrypt.HashPassword("password123");
    var user = new User { Username = "testuser", PasswordHash = passwordHash };

    _mockUnitOfWork.Setup(u => u.Users.GetByUsernameAsync("testuser"))
        .ReturnsAsync(user);

    // Act
    var result = await _userService.LoginAsync("testuser", "password123");

    // Assert
    result.Should().NotNull();
}
}

```

5.3. Kod Kalitesi - SonarQube Analizi

5.3.1. SonarQube Analiz Metrikleri

Proje kod tabanı, statik kod analizi aracı SonarQube ile taranmış ve aşağıdaki metrikler elde edilmiştir. Proje toplamda yaklaşık 16.000 satır (16k LoC) koddan oluşmaktadır.

Metrik	Açıklama	Değer	Durum
Reliability (Bugs)	Kod güvenilirliği ve hatalar	25	Grade E
Security (Vulnerabilities)	Güvenlik açıkları	13	Grade E
Maintainability (Code Smells)	Bakım kolaylığı ve kod kokuları	305	Grade A
Duplications	Kod tekrar oranı	%2.8	Grade A
Security Hotspots	Güvenlik açısından incelenmesi gereken noktalar	4	İnceleme Bekliyor
Lines of Code	Toplam kod satır sayısı	~16k	Bilgi
Test Coverage	Birim test kapsamı	-	Yapılandırılmadı

(Not: Geliştirme süreci devam ettiği için Reliability ve Security alanlarındaki bulgular backlog'a eklenmiş ve refactoring süreci planlanmıştır.)

5.3.2. Kod Kalitesi ve Güvenlik Özellikleri

Projede uygulanan teknik standartlar ve kalite güvence mekanizmaları aşağıdaki gibidir:

Özellik	Uygulama ve Teknoloji
Null Safety	C# Nullable Reference Types özelliği aktif edilerek NullReferenceException hataları minimize edilmiştir.
Exception Handling	Global hata yönetimi, try-catch blokları ve özelleştirilmiş CustomException sınıfları kullanılmıştır.
Logging & Audit	Kritik veri değişiklikleri ve hatalar için merkezi AuditLog sistemi kurulmuştur.
Validation	Kullanıcı girdileri FluentValidation kütüphanesi ile sunucu tarafında doğrulanmaktadır.
Security	Şifreler BCrypt ile hashlenmekte, hassas token verileri AES algoritması ile şifrelenmektedir.
Performance	Veritabanı işlemlerinde Async/Await pattern'i ve Lazy Loading (gerektiğinde) kullanılmıştır.

Harika, raporun sonuç kısmını akademik bir dille ve Word belgenize doğrudan kopyalayıp yapıştırabileceğiniz tablo formatında düzenledim.

6. SONUÇ

6.1. Projenin Genel Değerlendirmesi

Proje süreci sonunda elde edilen kazanımlar ve karşılaşılan kısıtlar aşağıda özetlenmiştir.

6.1.1. Proje Başarıları ve Güçlü Yönler

Başarı Faktörü	Açıklama ve Detaylar
Sağlam Mimari	5 katmanlı Enterprise mimari yapısı ve SOLID prensiplerine tam uyum.
Modern Teknolojiler	.NET 8.0, Entity Framework Core 8.0 ve C# 12.0 gibi en güncel teknolojilerin kullanımı.
Profesyonel UI	DevExpress kontrolleri ile tasarlanmış, Dark Theme destekli modern kullanıcı arayüzü.
GitHub Entegrasyonu	Commit senkronizasyonu ve görevlerle (Task) commit'lerin akıllı eşleştirilmesi.
Kapsamlı Test	177 adet Unit Test ile kod güvenilirliğinin sağlanması (%100 Başarı).
Dual-Database	Masaüstü ve Web arasındaki iletişim sorununun çift veritabanı ile çözülmesi.
E-posta Sistemi	Gmail SMTP altyapısı ile davet ve bilgilendirme maillerinin otomasyonu.
Raporlama	Detaylı analitik verilerin PDF ve Excel formatında dışa aktarılabilmesi.

6.1.2. Kısıtlar ve Eksiklikler

Eksiklik / Kısıt	Neden?	Çözüm Önceliği
Real-time Bildirimler	SignalR implementasyonu zaman kısıtı nedeniyle tamamlanamadı.	Yüksek
Mobil Uygulama	Proje kapsamı dışında bırakıldı (Gelecek faz).	Orta
Dosya Yükleme	Bulut veya yerel depolama altyapısı gereksinimi.	Orta
Gantt Chart	Gelişmiş Gantt şeması için ek lisans maliyeti.	Düşük

6.2. Projenin Bireysel Katkıları

Bu proje süreci, hem teknik yetkinliklerimi hem de süreç yönetimi becerilerimi önemli ölçüde geliştirmiştir.

6.2.1. Kazanılan Teknik Beceriler

Beceri Alanı	Öğrenilen ve Uygulanan Teknolojiler
Yazılım Mimarisi	Katmanlı Mimari (N-Tier), SOLID Prensipleri, Design Patterns (Repository, UoW).
Veri Erişimi (ORM)	Entity Framework Core, Code-First Yaklaşımı, Migration Yönetimi.
API Geliştirme	RESTful Servisler, ASP.NET Core Minimal API, CORS Politikaları.
Entegrasyon	GitHub REST API, SMTP Protokolü, Uzak Veritabanı Bağlantıları.
Test Mühendisliği	Unit Testing (xUnit), Mocking (Moq), Test Driven Development (TDD).
Arayüz (UI)	Windows Forms Geliştirme, DevExpress Kütüphanesi, UX Prensipleri.
Güvenlik	Veri Şifreleme (BCrypt, AES), Token Yönetimi.

6.2.2. Soft Skills (Kişisel Gelişim)

Beceri	Gelişim Açıklaması
Proje Yönetimi	Artırımlı geliştirme modelini uygulama ve zaman planlamasına sadık kalma.
Problem Çözme	Karşılaşılan mimari sorunlara (Dual-DB gibi) yaratıcı çözümler üretme.
Dokümantasyon	Standartlara uygun teknik dokümantasyon ve UML diyagramları hazırlama.
Araştırma	Literatür taraması ve yeni teknolojileri hızlı öğrenip adapte etme.

6.3. Proje Metrikleri Özeti

Projenin büyüklüğünü ve karmaşıklığını ifade eden sayısal veriler aşağıdadır.

Metrik	Değer
İşlev Noktası (İN)	639 (Çok Karmaşık)
Tahmini Kod Satırı (LoC)	~18,371
Geliştirme Süresi	~4-5 Ay
Unit Test Sayısı	177 (%100 Başarı)
Entity Sınıf Sayısı	18
Veritabanı Tablo Sayısı	18 (Local) + 1 (Remote)
Servis (Service) Sınıfı	14
Form ve Kontrol Sayısı	17
Veri Transfer Nesnesi (DTO)	27+

6.4. Gelecek Planları ve Yol Haritası

Projenin sürdürülebilirliği ve geliştirilmesi için planlanan sonraki adımlar:

Özellik / Geliştirme	Açıklama	Tahmini Süre
SignalR Entegrasyonu	Kullanıcılara anlık (real-time) bildirim gönderimi.	2-3 Hafta
Mobil Uygulama	.NET MAUI teknolojisi ile iOS ve Android desteği.	2-3 Ay
Gantt Chart Modülü	Proje zaman çizelgesinin görselleştirilmesi.	1-2 Hafta
Dosya Yönetim Sistemi	Proje ve görevlere dosya ekleme özelliği.	2-3 Hafta
Takvim Entegrasyonu	Görevlerin takvim üzerinde görüntülenmesi.	1 Hafta

7. KAYNAKLAR

7.1. Kitaplar

Projenin geliştirilme sürecinde başvurulmuş hem akademik hem de pratik kaynaklar aşağıdadır. "Clean Code" prensipleri için hem orijinal hem de Türkçe çeviri kaynaklardan yararlanılmıştır.

1. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall. (Orijinal Kaynak).
2. Martin, R. C. (Çev. Şahin, B.). Temiz Kod (Clean Code). Abaküs Kitap. (Türkçe Kaynak).
3. Yücedağ, M. (2020). Adım Adım C#. Kodlab Yayınları.
4. Yücedağ, M. (2022). ASP.NET Core MVC 5.0 ve Proje Uygulamaları. Kodlab Yayınları.
5. Albahari, J. (2022). C# 10 in a Nutshell. O'Reilly Media.
6. Smith, J. (2021). Entity Framework Core in Action. Manning Publications.

7.2. Online Dokümantasyon

1. **Microsoft Docs** - .NET Documentation: <https://docs.microsoft.com/dotnet>
2. **Entity Framework Core Documentation**: <https://docs.microsoft.com/ef/core>
3. **DevExpress Documentation**: <https://docs.devexpress.com>
4. **GitHub REST API Documentation**: <https://docs.github.com/rest>
5. **FluentValidation Documentation**: <https://docs.fluentvalidation.net>
6. **AutoMapper Documentation**: <https://docs.automapper.org>
7. **xUnit Documentation**: <https://xunit.net/docs>

7.3. Proje Bağlantıları ve Teknik Diyagramlar

Raporun ilgili bölümlerinde atıfta bulunulan diyagramların yüksek çözünürlüklü halleri ve proje kaynak kodları aşağıdaki bağlantılarda mevcuttur.

Kaynak Tipi	Açıklama	URL / Bağlantı
Kaynak Kod	Proje GitHub Repository	https://github.com/BilalAbic/ProjectTracker
Canlı Demo	Web Sitesi (Davet Sistemi)	https://pt.bilalabic.com
API	REST API Endpoint	https://bilalabic.com/api
Diyagram	Tam ER Diyagramı (Yüksek Çözünürlük)	ER Diyagram Drive Linki
Diyagram	Tüm UML Şemaları (Class, Use Case)	docs/UML/ (Repo içerisinde)

7.4. Eğitim Videoları ve Görsel Kaynaklar

Projenin mimari yapısı ve kodlama standartları oluşturulurken, alanında uzman eğitimcilerin yayınlarından faydalanılmıştır.

1. **Murat Yücedağ** - C# ve Kurumsal Mimari Eğitim Serileri (YouTube / Udemy).
2. **Gençay Yıldız** - .NET Core, Clean Architecture ve Design Patterns Eğitimleri (YouTube).

3. **Atıl Samancıoğlu** - Yazılım Geliştirme Prensipleri ve Programlama Temelleri (Udemy / Academy).
4. **Engin Demiroğ** - Yazılım Geliştirici Yetiştirme Kampı (C# & .NET) (YouTube).
5. **Nick Chapsas** - Modern .NET Development & Unit Testing (YouTube - İngilizce Kaynak).

7.5. Kullanılan NuGet Paketleri

Projede kullanılan üçüncü parti kütüphaneler ve versiyon bilgileri aşağıdadır.

Paket Adı	Versiyon	Kullanım Amacı
Microsoft.EntityFrameworkCore	8.0.0	Nesne-İlişkisel Eşleme (ORM)
Microsoft.EntityFrameworkCore.SqlServer	8.0.0	SQL Server Veritabanı Sağlayıcısı
AutoMapper	12.0.1	Nesne Dönüştürme (Object Mapping)
FluentValidation	11.9.0	Veri Doğrulama Kuralları
BCrypt.Net-Next	4.0.3	Güvenli Şifre Hashleme
Octokit	9.1.2	GitHub API İstemcisi
EPPlus	6.2.4	Excel Rapor Dışa Aktarma
iTextSharp	5.5.13.3	PDF Rapor Dışa Aktarma
DevExpress.WindowsForms	25.1.7	Profesyonel UI Kontrolleri
xUnit	2.5.3	Birim Test Çatısı
Moq	4.20.70	Test Mocking Kütüphanesi
FluentAssertions	6.12.0	Test Doğrulama (Assertions)